

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN
Môn: CƠ SỞ DỮ LIỆU PHÂN TÁN

Đề tài: Hệ thống bán hàng trực tuyến đa kho

Giảng viên hướng dẫn: Kim Ngọc Bách

Nhóm sinh viên thực hiện:

1. Nguyễn Trần Mai Anh - B23DCCN038
2. Nguyễn Trường Giang - B23DCCN258
3. Nguyễn Tiến Dũng - B23DCCN202

Hà Nội, Năm 2026

BẢNG PHÂN CÔNG NHIỆM VỤ THÀNH VIÊN

STT	Họ và tên	Nhiệm vụ	Chi tiết công việc
1	Nguyễn Trần Mai Anh	Chương 1 & 2	• Phân tích nghiệp vụ hệ thống bán hàng đa kho. • Thiết kế lược đồ quan hệ toàn cục và ràng buộc toàn vẹn. • Thực hiện phân mảnh cơ sở dữ liệu (ngang cơ sở, dọc, ngang dẫn xuất). • Biện luận chiến lược cấp phát và nhân bản dữ liệu.
2	Nguyễn Trường Giang	Chương 3	• Thiết kế và tối ưu hóa 5 câu truy vấn phân tán tiêu biểu. • Áp dụng kỹ thuật tối ưu phép nối nửa và tổng hợp cục bộ hai giai đoạn.
3	Nguyễn Tiến Dũng	Chương 4 & 5	• Thiết kế giao dịch phân tán và kiểm soát đồng thời (Strict 2PL, 2PC). • Cài đặt hệ thống phân tán vật lý trên 3 laptop qua Radmin VPN và Linked Server. • Xây dựng backend Node.js, giao diện Web demo và kiểm thử hệ thống.

Mục lục

Mở đầu	4
1 Tổng quan kiến trúc và thiết kế lược đồ toàn cục	6
1.1 Phân rã mạng lưới	6
1.1.1 Kiến trúc hệ thống phân tán	6
1.1.2 Xác định các trạm	7
1.1.3 Sơ đồ mạng lưới	8
1.2 Thiết kế lược đồ toàn cục	8
1.2.1 Phân tích thực thể và mối quan hệ	8
1.2.2 Sơ đồ thực thể – liên kết	10
1.2.3 Lược đồ quan hệ chi tiết	11
1.3 Kiểm soát toàn vẹn ngữ nghĩa phân tán	15
1.3.1 Ràng buộc khóa chính	15
1.3.2 Ràng buộc miền giá trị	16
1.3.3 Ràng buộc tham chiếu	16
1.3.4 Thách thức toàn vẹn trong môi trường phân tán	17
2 Thiết kế cơ sở dữ liệu phân tán	19
2.1 Thiết kế phân mảnh dữ liệu	19
2.1.1 Phân mảnh ngang cơ sở	19
2.1.2 Phân mảnh dọc bảng SANPHAM	22
2.1.3 Phân mảnh ngang dẫn xuất	24
2.1.4 Kiểm tra tính đúng đắn của phân mảnh	26
2.1.5 Tổng hợp kết quả phân mảnh	28
2.2 Chiến lược cấp phát và nhân bản	29
2.2.1 Biện luận chiến lược nhân bản	29
2.2.2 Sơ đồ cấp phát dữ liệu phân tán	30
3 Tối ưu hóa truy vấn phân tán	33
3.1 Tổng quan xử lý truy vấn phân tán	33
3.1.1 Các quy tắc rút gọn phân mảnh	33
3.1.2 Các kỹ thuật tối ưu hóa toàn cục	34
3.2 Phân tích và tối ưu 5 truy vấn nghiệp vụ	34

3.2.1	Truy vấn 1: Kiểm tra sản phẩm còn hàng tại kho miền Bắc	34
3.2.2	Truy vấn 2: Tính tổng tồn kho của một sản phẩm trên toàn hệ thống	36
3.2.3	Truy vấn 3: Tìm đơn hàng giao chéo vùng	37
3.2.4	Truy vấn 4: Thống kê doanh thu theo tháng của từng kho	40
3.2.5	Truy vấn 5: Tìm sản phẩm bán chạy nhất toàn hệ thống	42
3.3	Tổng hợp và đánh giá	44
4	Xử lý giao dịch và điều khiển đồng thời	46
4.1	Bài toán nhất quán tồn kho phân tán	46
4.1.1	Mô tả rủi ro	46
4.1.2	Giải pháp tổng quát	47
4.2	Kịch bản 1: đặt hàng cục bộ	48
4.2.1	Mô tả nghiệp vụ	48
4.2.2	Luồng thực thi	48
4.2.3	Điều khiển đồng thời bằng khóa hai pha cục bộ	48
4.2.4	Cài đặt SQL	48
4.3	Kịch bản 2: đặt hàng liên kho	50
4.3.1	Mô tả kịch bản	50
4.3.2	Điều khiển đồng thời bằng khóa hai pha nghiêm ngặt phân tán . .	50
4.3.3	Giao thức cam kết hai pha	52
4.3.4	Cài đặt SQL cho giao dịch phân tán	56
5	Triển khai và đánh giá hệ thống	60
5.1	Môi trường cài đặt và kiến trúc vật lý	60
5.1.1	Kiến trúc cơ sở dữ liệu phân tán	60
5.1.2	Tầng ứng dụng và giao diện quản trị	61
5.2	Đánh giá hệ thống	62
5.2.1	So sánh hiệu năng truy vấn cục bộ: tập trung và phân mảnh ngang	62
5.2.2	Ảnh hưởng của chiến lược nhân bản	63
5.2.3	Hiệu năng của phép nối nửa	63
5.2.4	Sự ổn định của giao dịch phân tán	64
5.3	Hướng dẫn chạy thử hệ thống	64
	Kết luận	67

Mở đầu

Bối cảnh bài toán

Trong mô hình kinh doanh thương mại điện tử hiện đại, các doanh nghiệp bán lẻ trực tuyến thường vận hành nhiều kho hàng phân bố tại các khu vực địa lý khác nhau nhằm rút ngắn thời gian giao hàng và giảm chi phí vận chuyển. Mỗi kho hoạt động như một đơn vị quản lý độc lập: tự theo dõi số lượng tồn kho, xử lý nhập xuất hàng, và đáp ứng đơn hàng trong phạm vi khu vực phụ trách.

Tuy nhiên, từ góc nhìn của khách hàng và ban quản trị, hệ thống phải hoạt động như một thể thống nhất. Khách hàng cần tra cứu tồn kho trên toàn bộ hệ thống, đặt hàng mà không cần biết sản phẩm được xuất từ kho nào, và theo dõi trạng thái đơn hàng xuyên suốt. Ban quản trị cần tổng hợp báo cáo doanh thu theo từng kho, từng khu vực, và toàn hệ thống.

Bài toán này đặt ra yêu cầu thiết kế một hệ thống cơ sở dữ liệu vừa đảm bảo tính tự chủ cục bộ tại mỗi kho, vừa duy trì khả năng phối hợp toàn cục — đây chính là đặc trưng cốt lõi của cơ sở dữ liệu phân tán.

Mục tiêu báo cáo

Báo cáo hướng tới các mục tiêu cụ thể sau:

1. Xây dựng mô hình cơ sở dữ liệu phân tán phản ánh đúng đặc trưng của hệ thống bán hàng nhiều kho, bao gồm lược đồ toàn cục, kiến trúc mạng lưới, và chiến lược phân bố dữ liệu.
2. Áp dụng các kỹ thuật phân mảnh dữ liệu (ngang cơ sở, ngang dẫn xuất) với biện luận học thuật dựa trên thuật toán COM_MIN và phân mảnh dẫn xuất bằng phép nối nửa.
3. Thiết kế chiến lược cấp phát và nhân bản dữ liệu giữa các trạm, có phân tích chi phí lưu trữ và truyền thông.
4. Xây dựng và tối ưu hóa các truy vấn phân tán phục vụ nghiệp vụ thực tế, sử dụng cây đại số quan hệ, kỹ thuật thu gọn và phép nối nửa.

5. Mô phỏng các giao dịch liên kho tiêu biểu, bao gồm cơ chế điều khiển đồng thời bằng khóa hai pha và cam kết phân tán bằng giao thức cam kết hai pha.
6. Triển khai chương trình demo minh họa hoạt động thực tế của hệ thống phân tán.

Phạm vi nghiên cứu

Báo cáo tập trung vào việc thiết kế và mô phỏng một hệ thống cơ sở dữ liệu phân tán cho doanh nghiệp thương mại điện tử có ba kho hàng tại ba khu vực: miền Bắc, miền Trung, và miền Nam. Hệ thống được xây dựng với các thực thể chính bao gồm: danh mục sản phẩm, sản phẩm, kho hàng, tồn kho, khách hàng, đơn hàng, và chi tiết đơn hàng.

Phạm vi kỹ thuật bao gồm:

- Thiết kế lược đồ toàn cục với ràng buộc toàn vẹn ngữ nghĩa.
- Phân mảnh ngang (cơ sở và dẫn xuất) và biện luận tính đúng đắn.
- Chiến lược cấp phát và nhân bản dữ liệu.
- Xử lý và tối ưu hóa truy vấn phân tán.
- Điều khiển đồng thời và đảm bảo nhất quán dữ liệu.
- Triển khai demo trên nền tảng web.

Chương 1

Tổng quan kiến trúc và thiết kế lược đồ toàn cục

Chương này trình bày nền tảng kiến trúc của hệ thống cơ sở dữ liệu phân tán, bao gồm: mô hình mạng lưới các trạm, lược đồ toàn cục với đầy đủ các thực thể và mối quan hệ, cùng hệ thống ràng buộc toàn vẹn ngữ nghĩa. Đây là bước thiết kế nền móng, quyết định toàn bộ chiến lược phân mảnh, cấp phát và xử lý truy vấn ở các chương tiếp theo.

1.1 Phân rã mạng lưới

1.1.1 Kiến trúc hệ thống phân tán

Hệ thống bán hàng trực tuyến đa kho được xây dựng trên kiến trúc nhiều server cơ sở dữ liệu, trong đó mỗi server (gọi là *trạm*) đại diện cho một kho hàng tại một khu vực địa lý cụ thể. Các trạm được kết nối với nhau thông qua phân hệ truyền thông, tạo thành một hệ quản trị cơ sở dữ liệu phân tán thống nhất.

Kiến trúc này mang lại ba đặc tính thiết yếu:

- **Tính tự chủ cục bộ:** Mỗi trạm có khả năng xử lý độc lập các nghiệp vụ liên quan đến kho hàng của mình — bao gồm nhập xuất hàng, quản lý tồn kho, và xử lý đơn hàng cục bộ — mà không phụ thuộc vào các trạm khác. Điều này giúp giảm thiểu lưu lượng truyền thông liên trạm trong các thao tác thường ngày.
- **Tính trong suốt:** Người dùng và ứng dụng tương tác với hệ thống như thể đang làm việc với một cơ sở dữ liệu tập trung duy nhất. Mục tiêu này đạt được thông qua lược đồ toàn cục, giúp che giấu sự phân tán vật lý của dữ liệu. Cụ thể, hệ thống cung cấp ba mức trong suốt:

– *Trong suốt phân mảnh:* Người dùng truy vấn trên quan hệ toàn cục mà không cần biết dữ liệu đã được chia thành các mảnh.

- *Trong suốt vị trí:* Người dùng không cần chỉ định dữ liệu nằm ở trạm nào.
- *Trong suốt ánh xạ cục bộ:* Mỗi trạm có thể sử dụng hệ quản trị cơ sở dữ liệu riêng mà không ảnh hưởng đến truy vấn toàn cục.
- **Tính sẵn sàng cao:** Khi một trạm gặp sự cố, các trạm còn lại vẫn hoạt động bình thường trong phạm vi dữ liệu cục bộ. Dữ liệu dùng chung (như danh mục sản phẩm) được nhân bản tại nhiều trạm, đảm bảo khả năng truy cập ngay cả khi mạng bị phân đoạn.

Mô hình nhiều server phù hợp với đặc thù nghiệp vụ của doanh nghiệp thương mại điện tử đa kho. Phần lớn các giao dịch hàng ngày — kiểm tra tồn kho, xử lý xuất kho, cập nhật số lượng — diễn ra tại cùng một kho, tức cùng một trạm. Các truy vấn liên trạm (tra cứu tồn kho toàn hệ thống, thống kê doanh thu tổng hợp, đặt hàng liên kho khi kho cục bộ không đủ hàng) chiếm tỷ lệ nhỏ hơn nhưng được hỗ trợ đầy đủ thông qua phân hệ truyền thông. Tỷ lệ nghiệp vụ cục bộ so với liên trạm cao là điều kiện lý tưởng để kiến trúc phân tán phát huy hiệu quả.

1.1.2 Xác định các trạm

Dựa trên chiến lược chia để trị theo khu vực địa lý, hệ thống được phân rã thành ba trạm, mỗi trạm tương ứng với một khu vực kinh doanh chính của doanh nghiệp.

Bảng 1.1: Danh sách các trạm trong hệ thống phân tán

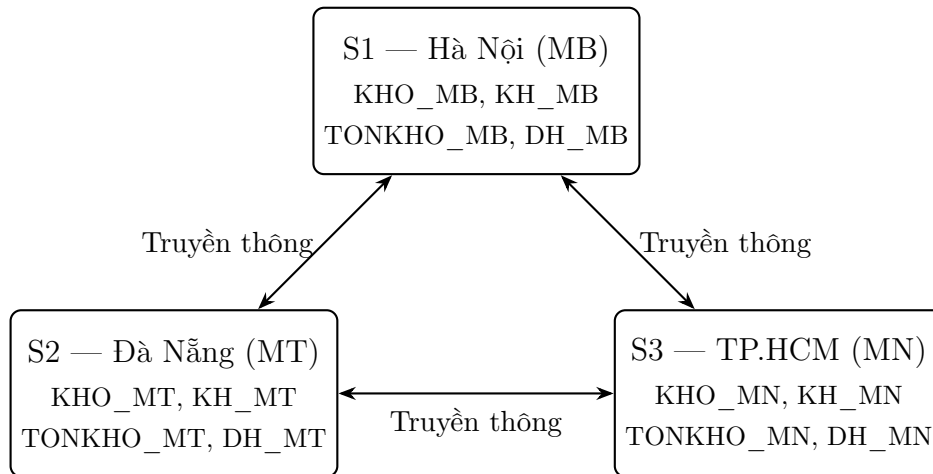
Trạm	Vị trí đặt server	Mã khu vực	Phạm vi quản lý
S1	Hà Nội	MB	Các kho và khách hàng miền Bắc
S2	Đà Nẵng	MT	Các kho và khách hàng miền Trung
S3	TP. Hồ Chí Minh	MN	Các kho và khách hàng miền Nam

Việc phân chia theo ba miền Bắc – Trung – Nam phản ánh thực tế phân bố kho hàng và khách hàng của doanh nghiệp. Chiến lược này đảm bảo nguyên tắc cục bộ hóa dữ liệu: dữ liệu được truy cập thường xuyên nhất bởi một khu vực sẽ được lưu trữ tại trạm phục vụ khu vực đó. Cụ thể, mỗi trạm chịu trách nhiệm:

- (a) Lưu trữ và quản lý dữ liệu các kho hàng thuộc khu vực tương ứng (bảng KHO với KhuVuc phù hợp).
- (b) Quản lý thông tin khách hàng đăng ký tại khu vực (bảng KHACHHANG với KhuVuc phù hợp).
- (c) Lưu trữ dữ liệu tồn kho của các kho trong khu vực (bảng TONKHO liên kết với các kho cục bộ).

- (d) Xử lý đơn hàng xuất phát từ khách hàng khu vực (bảng DONHANG liên kết với khách hàng cục bộ) và phần chi tiết đơn hàng xuất từ kho cục bộ (bảng CHITIETDONHANG với MaKhoXuat thuộc khu vực).

1.1.3 Sơ đồ mạng lưới



Hình 1.1: Sơ đồ mạng lưới ba trạm kết nối đầy đủ

Sơ đồ mạng lưới minh họa kiến trúc vật lý của hệ thống với ba trạm kết nối theo mô hình đầy đủ: mỗi trạm có thể giao tiếp trực tiếp với bất kỳ trạm nào khác thông qua phân hệ truyền thông. Topology này đảm bảo:

- Không có điểm nghẽn trung tâm — tránh phụ thuộc vào một trạm trung gian duy nhất.
- Độ trễ truyền thông tối thiểu cho các truy vấn liên trạm — dữ liệu được truyền trực tiếp giữa hai trạm liên quan mà không cần chuyển tiếp.
- Hỗ trợ hiệu quả cho các giao dịch liên kho cần phối hợp giữa hai hoặc ba trạm đồng thời (ví dụ: đặt hàng liên kho cần kiểm tra tồn kho ở nhiều trạm).

1.2 Thiết kế lược đồ toàn cục

Lược đồ toàn cục định nghĩa cấu trúc logic thống nhất của toàn bộ cơ sở dữ liệu, bất kể dữ liệu thực sự được lưu trữ ở trạm nào. Đây là tầng trừu tượng cao nhất trong kiến trúc cơ sở dữ liệu phân tán, đóng vai trò cung cấp tính trong suốt cho người dùng và ứng dụng.

1.2.1 Phân tích thực thể và mối quan hệ

Từ yêu cầu nghiệp vụ của hệ thống bán hàng đa kho, ta xác định được sáu thực thể chính và các mối quan hệ giữa chúng.

Danh sách thực thể

Bảng 1.2: Các thực thể trong lược đồ toàn cục

STT	Thực thể	Mô tả
1	DANHMUC	Nhóm phân loại sản phẩm (ví dụ: điện tử, gia dụng, thời trang). Dữ liệu tĩnh, ít thay đổi, được toàn bộ hệ thống sử dụng.
2	SANPHAM	Thông tin chi tiết về từng sản phẩm: tên, giá bán, danh mục. Dữ liệu dùng chung, cần thiết tại mọi trạm để xử lý đơn hàng và hiển thị cho khách.
3	KHO	Thông tin các kho hàng vật lý: tên, địa chỉ, khu vực. Mỗi kho thuộc đúng một khu vực và được quản lý bởi trạm tương ứng.
4	TONKHO	Số lượng tồn kho của từng sản phẩm tại từng kho. Đây là thực thể kết hợp giữa KHO và SANPHAM, chứa dữ liệu biến động cao nhất trong hệ thống.
5	KHACHHANG	Thông tin khách hàng: tên, khu vực, số điện thoại. Mỗi khách hàng được gán với một khu vực và quản lý bởi trạm tương ứng.
6	DONHANG	Đơn đặt hàng của khách, ghi nhận thời điểm đặt và trạng thái xử lý. Mỗi đơn thuộc về một khách hàng cụ thể.
7	CHITIETDONHANG	Chi tiết từng dòng sản phẩm trong đơn hàng, bao gồm kho xuất hàng. Cho phép một đơn hàng được đáp ứng từ nhiều kho khác nhau.

Các mối quan hệ

Các mối quan hệ giữa các thực thể tuân theo mô hình chủ – thành viên, trong đó thực thể chủ nắm giữ khóa chính được tham chiếu bởi thực thể thành viên thông qua khóa ngoại.

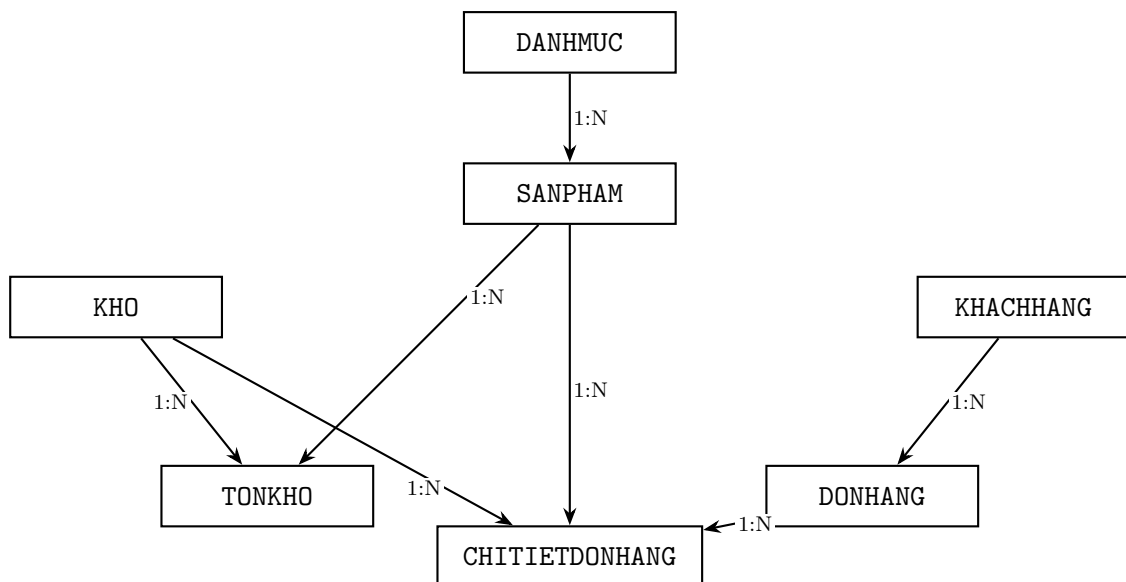
Bảng 1.3: Các mối quan hệ trong lược đồ toàn cục

Mối quan hệ	Loại	Thuộc tính liên kết	Ý nghĩa nghiệp vụ
DANHMUC → SANPHAM	1:N	MaDM	Một danh mục chứa nhiều sản phẩm
KHO → TONKHO	1:N	MaKho	Một kho lưu trữ nhiều loại sản phẩm
SANPHAM → TONKHO	1:N	MaSP	Một sản phẩm tồn tại ở nhiều kho
KHACHHANG → DONHANG	1:N	MaKH	Một khách hàng đặt nhiều đơn
DONHANG → CHITIETDONHANG	1:N	MaDH	Một đơn có nhiều dòng chi tiết
SANPHAM → CHITIETDONHANG	1:N	MaSP	Một sản phẩm xuất hiện ở nhiều đơn
KHO → CHITIETDONHANG	1:N	MaKhoXuat	Một kho xuất hàng cho nhiều đơn

Điểm đáng chú ý trong thiết kế quan hệ là bảng **CHITIETDONHANG** có ba thực thể chủ đồng thời: **DONHANG**, **SANPHAM**, và **KHO**. Cấu trúc này phản ánh yêu cầu nghiệp vụ: cùng một dòng sản phẩm trong một đơn hàng có thể được xuất từ các kho khác nhau (khi kho cục bộ không đủ hàng). Khóa chính tổ hợp (**MaDH**, **MaSP**, **MaKhoXuat**) cho phép ghi nhận chính xác việc phân bổ hàng từ nhiều kho cho cùng một đơn.

Cấu trúc “lai” này tạo ra thách thức lớn cho bài toán phân mảnh dẫn xuất ở chương 2: **CHITIETDONHANG** vừa phụ thuộc vào **DONHANG** (phân mảnh theo khu vực khách hàng) vừa phụ thuộc vào **KHO** (phân mảnh theo khu vực kho). Việc lựa chọn thực thể chủ nào làm cơ sở cho phân mảnh dẫn xuất sẽ ảnh hưởng trực tiếp đến hiệu quả truy vấn.

1.2.2 Sơ đồ thực thể – liên kết

**Hình 1.2:** Sơ đồ thực thể — liên kết toàn cục

Sơ đồ thực thể – liên kết minh họa cấu trúc logic của lược đồ toàn cục. Các mối quan hệ chủ – thành viên được thể hiện bằng đường nối có ký hiệu lực lượng (1:N). Các thuộc tính khóa chính được gạch chân để phân biệt với thuộc tính thường.

Từ sơ đồ, ta nhận thấy hai nhóm thực thể có đặc điểm phân tán khác nhau:

- **Nhóm dùng chung:** DANHMUC và SANPHAM — dữ liệu tĩnh hoặc ít thay đổi, cần thiết tại mọi trạm. Đây là ứng viên cho chiến lược nhân bản toàn phần.
- **Nhóm phân vùng:** KHO, KHACHHANG, TONKHO, DONHANG, CHITTIETDONHANG — dữ liệu có tính cục bộ cao theo khu vực. Đây là ứng viên cho phân mảnh ngang theo thuộc tính KhuVuc hoặc theo liên kết với thực thể chủ.

1.2.3 Lược đồ quan hệ chi tiết

Phần này trình bày chi tiết cấu trúc từng bảng trong lược đồ toàn cục, bao gồm: lược đồ quan hệ, kiểu dữ liệu, ràng buộc, và giải thích thiết kế.

Bảng DANHMUC

Lược đồ quan hệ: DANHMUC(MaDM, TenDM, MoTa)

Bảng 1.4: Cấu trúc bảng DANHMUC

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
MaDM	VARCHAR(10)	PRIMARY KEY	Mã định danh danh mục
TenDM	NVARCHAR(100)	NOT NULL	Tên danh mục sản phẩm
MoTa	NVARCHAR(255)		Mô tả chi tiết danh mục

Vai trò trong hệ phân tán: Bảng DANHMUC chứa dữ liệu tham chiếu dùng chung, ít thay đổi. Mọi trạm đều cần truy cập danh mục để phân loại sản phẩm và hiển thị giao diện người dùng. Do đó, bảng này là ứng viên hàng đầu cho chiến lược nhân bản toàn phần tại tất cả các trạm (phân tích chi phí cụ thể ở chương 2).

Bảng SANPHAM

Lược đồ quan hệ: SANPHAM(MaSP, TenSP, GiaBan, MaDM)

Bảng 1.5: Cấu trúc bảng SANPHAM

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
MaSP	VARCHAR(10)	PRIMARY KEY	Mã định danh sản phẩm
TenSP	NVARCHAR(100)	NOT NULL	Tên sản phẩm
GiaBan	DECIMAL(18,2)	CHECK (GiaBan > 0)	Giá bán lẻ sản phẩm
MaDM	VARCHAR(10)	FOREIGN KEY → DANHMUC	Mã danh mục chứa sản phẩm

Vai trò trong hệ phân tán: Tương tự DANHMUC, thông tin sản phẩm cần thiết tại mọi trạm để xử lý đơn hàng, hiển thị catalog, và tính toán doanh thu. Ràng buộc CHECK

(GiaBan > 0) đảm bảo tính hợp lệ của dữ liệu ngay tại mức lược đồ, tránh phải kiểm tra tại tầng ứng dụng.

Bảng KHO

Lược đồ quan hệ: KHO(MaKho, TenKho, KhuVuc, DiaChi)

Bảng 1.6: Cấu trúc bảng KHO

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
MaKho	VARCHAR(10)	PRIMARY KEY	Mã định danh kho hàng
TenKho	NVARCHAR(100)	NOT NULL	Tên kho hàng
KhuVuc	VARCHAR(2)	CHECK (KhuVuc IN ('MB', 'MT', 'MN'))	Mã khu vực (miền Bắc, Trung, Nam)
DiaChi	NVARCHAR(255)		Địa chỉ chi tiết của kho

Vai trò trong hệ phân tán: Bảng KHO là bảng nền tảng cho chiến lược phân mảnh ngang. Thuộc tính KhuVuc chính là thuộc tính phân mảnh — dữ liệu được chia thành ba mảnh ngang cơ sở dựa trên giá trị KhuVuc:

- $KHO_1 = \sigma_{KhuVuc='MB'}(KHO)$ — chứa các kho miền Bắc, lưu tại S1.
- $KHO_2 = \sigma_{KhuVuc='MT'}(KHO)$ — chứa các kho miền Trung, lưu tại S2.
- $KHO_3 = \sigma_{KhuVuc='MN'}(KHO)$ — chứa các kho miền Nam, lưu tại S3.

Ràng buộc CHECK trên KhuVuc đảm bảo rằng mỗi kho chỉ thuộc một trong ba khu vực hợp lệ, hỗ trợ tính tách biệt khi phân mảnh.

Bảng TONKHO

Lược đồ quan hệ: TONKHO(MaKho, MaSP, SoLuongTon)

Bảng 1.7: Cấu trúc bảng TONKHO

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
MaKho	VARCHAR(10)	PK, FK → KHO	Mã kho hàng
MaSP	VARCHAR(10)	PK, FK → SANPHAM	Mã sản phẩm
SoLuongTon	INT	CHECK (SoLuongTon ≥ 0)	Số lượng hiện có tại kho

Vai trò trong hệ phân tán: TONKHO là bảng chứa dữ liệu biến động nhất trong hệ thống — mỗi lần xuất hàng, nhập hàng, hay điều chuyển đều làm thay đổi SoLuongTon. Ràng buộc CHECK (SoLuongTon ≥ 0) là tuyến phòng thủ cuối cùng chống tình trạng âm kho, bổ sung cho cơ chế khóa hai pha ở tầng giao dịch (chương 4).

Bảng này là **thực thể thành viên** của cả KHO và SANPHAM. Do KHO được phân mảnh ngang theo KhuVuc, bảng TONKHO sẽ được phân mảnh ngang dẫn xuất dựa trên phép nối nửa với KHO: mỗi trạm chỉ lưu dữ liệu tồn kho của các kho thuộc khu vực mình.

Bảng KHACHHANG

Lược đồ quan hệ: KHACHHANG(MaKH, TenKH, KhuVuc, DiaChi, SoDienThoai)

Bảng 1.8: Cấu trúc bảng KHACHHANG

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
MaKH	VARCHAR(10)	PRIMARY KEY	Mã định danh khách hàng
TenKH	NVARCHAR(100)	NOT NULL	Họ tên khách hàng
KhuVuc	VARCHAR(2)	CHECK (KhuVuc IN ('MB', 'MT', 'MN'))	Khu vực đăng ký của khách
DiaChi	NVARCHAR(255)		Địa chỉ của khách hàng
SoDienThoai	VARCHAR(15)		Số điện thoại liên hệ

Vai trò trong hệ phân tán: Tương tự bảng KHO, bảng KHACHHANG được phân mảnh ngang cơ sở theo thuộc tính KhuVuc. Mỗi trạm lưu trữ thông tin khách hàng thuộc khu vực mình phục vụ. Chiến lược phân mảnh này đảm bảo rằng phần lớn truy vấn liên quan đến khách hàng (đăng nhập, xem lịch sử mua hàng, cập nhật thông tin) được xử lý cục bộ tại trạm quản lý khu vực đó.

- $KHACHHANG_1 = \sigma_{KhuVuc='MB'}(KHACHHANG)$ — lưu tại S1.
- $KHACHHANG_2 = \sigma_{KhuVuc='MT'}(KHACHHANG)$ — lưu tại S2.
- $KHACHHANG_3 = \sigma_{KhuVuc='MN'}(KHACHHANG)$ — lưu tại S3.

Bảng DONHANG

Lược đồ quan hệ: DONHANG(MaDH, MaKH, NgayLap, TrangThai)

Bảng 1.9: Cấu trúc bảng DONHANG

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
MaDH	VARCHAR(10)	PRIMARY KEY	Mã định danh đơn hàng
MaKH	VARCHAR(10)	FK → KHACHHANG	Mã khách hàng đặt đơn
NgayLap	DATETIME	DEFAULT hiện tại	Ngày giờ tạo đơn hàng
TrangThai	NVARCHAR(50)	DEFAULT 'ChoXuLy'	Trạng thái xử lý đơn

Biện luận thiết kế — loại bỏ thuộc tính TongTien:

Một quyết định kiến trúc quan trọng trong thiết kế bảng DONHANG là không lưu trữ thuộc tính dẫn xuất TongTien. Lý do như sau:

Trong hệ thống đa kho, một đơn hàng có thể được đáp ứng từ nhiều kho khác nhau. Các dòng chi tiết đơn hàng (CHITIETDONHANG) với MaKhoXuat khác nhau sẽ nằm ở các trạm khác nhau. Nếu lưu TongTien tại bảng DONHANG, mỗi lần thêm hoặc sửa một dòng chi tiết sẽ kích hoạt giao thức cam kết hai pha để cập nhật đồng bộ giá trị TongTien — một thao tác tốn kém về mặt truyền thông và phức tạp về mặt đồng bộ hóa.

Thay vào đó, tổng tiền của đơn hàng được tính toán tại thời điểm truy vấn bằng phép tổng hợp:

```
1 SELECT dh.MaDH, SUM(ct.SoLuong * ct.DonGia) AS TongTien
2 FROM DONHANG dh
3 JOIN CHITIETDONHANG ct ON dh.MaDH = ct.MaDH
4 GROUP BY dh.MaDH;
```

Đoạn mã 1.1: Truy vấn tính tổng tiền đơn hàng

Giải pháp này chuyển chi phí từ *giao dịch cập nhật phân tán* (tốn kém, cần 2PC) sang *truy vấn tổng hợp phân tán* (nhẹ hơn, chỉ cần đọc). Đây là sự đánh đổi hợp lý trong môi trường phân tán: ưu tiên giảm thiểu giao dịch ghi liên trạm, chấp nhận tăng nhẹ chi phí đọc.

Vai trò trong hệ phân tán: DONHANG là thực thể thành viên của KHACHHANG. Do KHACHHANG được phân mảnh theo KhuVuc, bảng DONHANG sẽ được phân mảnh ngang dẫn xuất theo khách hàng: đơn hàng của khách miền Bắc lưu tại S1, miền Trung lưu tại S2, miền Nam lưu tại S3.

Bảng CHITIETDONHANG

Lược đồ quan hệ: CHITIETDONHANG(MaDH, MaSP, MaKhoXuat, SoLuong, DonGia)

Bảng 1.10: Cấu trúc bảng CHITIETDONHANG

Thuộc tính	Kiểu dữ liệu	Ràng buộc	Mô tả
MaDH	VARCHAR(10)	PK, FK → DONHANG	Mã đơn hàng
MaSP	VARCHAR(10)	PK, FK → SANPHAM	Mã sản phẩm
MaKhoXuat	VARCHAR(10)	PK, FK → KHO	Mã kho xuất hàng
SoLuong	INT	CHECK (SoLuong > 0)	Số lượng đặt mua
DonGia	DECIMAL(18,2)	CHECK (DonGia ≥ 0)	Đơn giá tại thời điểm đặt

Phân tích cấu trúc “lai”:

Bảng CHITIETDONHANG là bảng phức tạp nhất trong lược đồ toàn cục vì mang đặc tính của **bảng lai** — có đồng thời nhiều thực thể chủ nằm ở các trạm khác nhau:

- **Thực thể chủ 1** — DONHANG: Phân mảnh theo khu vực khách hàng. Đơn hàng của khách miền Bắc nằm ở S1, do đó chi tiết đơn cũng “nên” ở S1.

- **Thực thể chủ 2 — KHO (qua MaKhoXuat):** Phân mảnh theo khu vực kho. Kho miền Nam nằm ở S3, do đó chi tiết xuất từ kho miền Nam “nên” ở S3.

Khi một khách hàng miền Bắc (S1) đặt hàng nhưng sản phẩm được xuất từ kho miền Nam (S3), hai thực thể chủ “kéo” dòng chi tiết về hai trạm khác nhau. Đây là xung đột phân mảnh dẫn xuất điển hình, sẽ được giải quyết trong chương 2 bằng cách phân tích tần suất truy vấn để chọn thực thể chủ phù hợp nhất.

Ý nghĩa của khóa chính tổ hợp (MaDH, MaSP, MaKhoXuat):

Khóa chính ba thành phần cho phép ghi nhận tình huống: cùng một sản phẩm trong cùng một đơn hàng được xuất từ nhiều kho khác nhau. Ví dụ: khách đặt 10 sản phẩm A, kho Hà Nội có 4, kho Đà Nẵng có 6 — sẽ sinh ra hai bản ghi CHITIETDONHANG riêng biệt với cùng MaDH và MaSP nhưng khác MaKhoXuat.

Thuộc tính DonGia: Giá bán tại thời điểm đặt hàng được lưu trực tiếp trong chi tiết đơn (thay vì tham chiếu từ bảng SANPHAM) để đảm bảo tính chính xác lịch sử: khi giá sản phẩm thay đổi, các đơn hàng cũ vẫn phản ánh đúng giá tại thời điểm giao dịch.

1.3 Kiểm soát toàn vẹn ngữ nghĩa phân tán

Trong môi trường cơ sở dữ liệu phân tán, việc kiểm soát toàn vẹn dữ liệu trở nên phức tạp hơn so với hệ tập trung do dữ liệu được phân tán trên nhiều trạm. Các ràng buộc toàn vẹn phải được thiết lập ngay tại mức lược đồ toàn cục để đảm bảo tính nhất quán bất kể dữ liệu nằm ở trạm nào.

1.3.1 Ràng buộc khóa chính

Khóa chính đảm bảo mỗi bản ghi trong quan hệ là duy nhất. Trong hệ phân tán, ràng buộc này phải được duy trì trên phạm vi toàn cục — không được phép có hai bản ghi cùng khóa chính ở hai trạm khác nhau.

Bảng 1.11: Tổng hợp khóa chính trong lược đồ toàn cục

Bảng	Khóa chính	Ghi chú
DANHMUC	MaDM	Khóa đơn, nhân bản tại mọi trạm
SANPHAM	MaSP	Khóa đơn, nhân bản tại mọi trạm
KHO	MaKho	Khóa đơn, phân mảnh ngang
TONKHO	(MaKho, MaSP)	Khóa tổ hợp, xác định duy nhất cặp kho-sản phẩm
KHACHHANG	MaKH	Khóa đơn, phân mảnh ngang
DONHANG	MaDH	Khóa đơn, phân mảnh dẫn xuất
CHITIETDONHANG	(MaDH, MaSP, MaKhoXuat)	Khóa tổ hợp ba thành phần

Đối với các bảng được phân mảnh ngang (mỗi mảnh ở một trạm), tính duy nhất khóa chính được đảm bảo bởi hai yếu tố phối hợp:

- (1) **Tính tách biệt của phân mảnh:** Các mảnh ngang không chồng lấn (chứng minh ở chương 2), do đó một bản ghi chỉ thuộc đúng một mảnh tại đúng một trạm.
- (2) **Quy ước sinh mã:** Mã định danh (MaKho, MaKH, MaDH) được sinh theo quy tắc có tiền tố khu vực (ví dụ: KH_MB001 cho kho miền Bắc), tránh trùng lặp giữa các trạm.

1.3.2 Ràng buộc miền giá trị

Ràng buộc miền giới hạn tập giá trị hợp lệ của từng thuộc tính, được cài đặt bằng mệnh đề CHECK trong câu lệnh tạo bảng.

Bảng 1.12: Tổng hợp ràng buộc miền giá trị

Bảng	Ràng buộc CHECK	Mục đích
SANPHAM	GiaBan > 0	Giá bán phải dương, ngăn lỗi nhập liệu
KHO	KhuVuc IN ('MB','MT','MN')	Chỉ cho phép ba mã khu vực hợp lệ, hỗ trợ phân mảnh
TONKHO	SoLuongTon ≥ 0	Chống âm kho — tuyến phòng thủ cuối cùng ở mức CSDL
KHACHHANG	KhuVuc IN ('MB','MT','MN')	Tương tự KHO, đảm bảo tính hợp lệ khu vực khách
CHITIETDONHANG	SoLuong > 0	Số lượng đặt phải dương
CHITIETDONHANG	DonGia ≥ 0	Đơn giá không âm (cho phép giá 0 trong trường hợp khuyến mãi)

Ràng buộc CHECK ($SoLuongTon \geq 0$) trên bảng TONKHO đặc biệt quan trọng trong hệ phân tán. Khi có nhiều đơn hàng đồng thời cùng yêu cầu xuất hàng từ một kho, cơ chế khóa hai pha ở tầng giao dịch (chương 4) sẽ tuần tự hóa các thao tác cập nhật. Tuy nhiên, ràng buộc CHECK tại mức cơ sở dữ liệu đóng vai trò lưới an toàn bổ sung: ngay cả khi tầng ứng dụng mắc lỗi logic, hệ quản trị sẽ từ chối bất kỳ cập nhật nào khiến $SoLuongTon$ xuống dưới 0.

1.3.3 Ràng buộc tham chiếu

Ràng buộc tham chiếu (khóa ngoại) đảm bảo tính nhất quán liên bảng: giá trị khóa ngoại tại bảng thành viên phải tồn tại trong bảng chủ.

Bảng 1.13: Tổng hợp ràng buộc tham chiếu (khóa ngoại)

Bảng thành viên	Khóa ngoại	Bảng chủ (tham chiếu)
SANPHAM	MaDM	DANHMUC(MaDM)
TONKHO	MaKho	KHO(MaKho)
TONKHO	MaSP	SANPHAM(MaSP)
DONHANG	MaKH	KHACHHANG(MaKH)
CHITIETDONHANG	MaDH	DONHANG(MaDH)
CHITIETDONHANG	MaSP	SANPHAM(MaSP)
CHITIETDONHANG	MaKhoXuat	KHO(MaKho)

1.3.4 Thách thức toàn vẹn trong môi trường phân tán

Trong hệ tập trung, tất cả ràng buộc tham chiếu được kiểm tra cục bộ vì toàn bộ dữ liệu nằm trên một server. Trong hệ phân tán, một số ràng buộc trở thành **ràng buộc liên trạm** — bảng chủ và bảng thành viên có thể nằm ở các trạm khác nhau.

Phân loại ràng buộc theo phạm vi:

Bảng 1.14: Phân loại ràng buộc tham chiếu theo phạm vi

Ràng buộc	Phạm vi	Giải thích
SANPHAM.MaDM → DANHMUC	Cục bộ	Cả hai bảng đều được nhân bản, cùng tồn tại tại mọi trạm
TONKHO.MaKho → KHO	Cục bộ	TONKHO phân mảnh dẫn xuất theo KHO, cùng trạm với kho chủ
TONKHO.MaSP → SANPHAM	Cục bộ	SANPHAM nhân bản, luôn có bản sao tại trạm chứa TONKHO
DONHANG.MaKH → KHACHHANG	Cục bộ	DONHANG phân mảnh dẫn xuất theo KHACHHANG, cùng trạm
CTDH.MaDH → DONHANG	Phụ thuộc chiến lược	Nếu phân mảnh theo DONHANG: cục bộ. Nếu theo KHO: có thể liên trạm
CTDH.MaSP → SANPHAM	Cục bộ	SANPHAM nhân bản tại mọi trạm
CTDH.MaKhoXuat → KHO	Phụ thuộc chiến lược	Nếu phân mảnh theo KHO: cục bộ. Nếu theo DONHANG: có thể liên trạm

Bảng trên cho thấy phần lớn ràng buộc tham chiếu đều có thể được kiểm tra cục bộ nhờ hai yếu tố:

- (1) **Chiến lược nhân bản:** Các bảng dùng chung (DANHMUC, SANPHAM) được nhân bản tại mọi trạm, do đó bất kỳ tham chiếu nào đến chúng đều có thể kiểm tra cục bộ.

- (2) **Phân mảnh dẫn xuất đồng nhất:** Bảng thành viên (TONKHO, DONHANG) được phân mảnh dẫn xuất theo cùng tiêu chí với bảng chủ, đảm bảo hai bên cùng trạm.

Trường hợp ngoại lệ duy nhất là bảng CHITIETDONHANG — do có hai thực thể chủ ở phân mảnh khác nhau, **luôn có ít nhất một ràng buộc tham chiếu trở thành liên trạm** bất kể chọn chiến lược phân mảnh nào. Đây là sự đánh đổi không thể tránh khỏi và cần được xử lý ở tầng giao dịch (chương 4) thay vì tầng lược đồ.

Tổng kết chương 1

Chương này đã thiết lập nền tảng kiến trúc cho hệ thống cơ sở dữ liệu phân tán:

- Mạng lưới ba trạm (S1, S2, S3) theo kiến trúc nhiều server, kết nối đầy đủ.
- Lược đồ toàn cục gồm 7 bảng với đầy đủ ràng buộc khóa, miền, và tham chiếu.
- Quyết định loại bỏ thuộc tính dẫn xuất **TongTien** để tối ưu giao dịch phân tán.
- Nhận diện bảng CHITIETDONHANG là bảng lai với xung đột phân mảnh dẫn xuất.
- Phân tích phạm vi các ràng buộc toàn vẹn (cục bộ và liên trạm).

Các kết quả thiết kế trong chương này sẽ được sử dụng trực tiếp ở chương 2 để tiến hành phân mảnh dữ liệu bằng thuật toán COM_MIN và xây dựng chiến lược cấp phát, nhân bản.

Chương 2

Thiết kế cơ sở dữ liệu phân tán

Chương này trình bày chiến lược thiết kế cơ sở dữ liệu phân tán cho hệ thống bán hàng đa kho thông qua việc áp dụng các kỹ thuật phân mảnh dữ liệu (ngang cơ sở, dọc, ngang dẫn xuất) và xây dựng chiến lược cấp phát, nhân bản. Mục tiêu tổng quát là tối ưu hóa hàm chi phí toàn cục bao gồm: chi phí lưu trữ, chi phí truyền thông liên trạm, và chi phí xử lý truy vấn — đồng thời đảm bảo tính cục bộ cao nhất cho các nghiệp vụ thường xuyên.

2.1 Thiết kế phân mảnh dữ liệu

Hệ thống vận hành trên ba vùng địa lý (miền Bắc, miền Trung, miền Nam) với ba trạm tương ứng (S1, S2, S3). Dựa trên đặc thù truy xuất dữ liệu, hệ thống áp dụng phối hợp ba kỹ thuật phân mảnh:

- **Phân mảnh ngang cơ sở:** Chia quan hệ thành các tập con bản ghi dựa trên vị từ chọn. Áp dụng cho các quan hệ độc lập có thuộc tính phân vùng rõ ràng (KHO, KHACHHANG).
- **Phân mảnh dọc:** Chia quan hệ thành các tập con thuộc tính dựa trên tần suất truy xuất đồng thời. Áp dụng cho quan hệ có nhóm thuộc tính truy xuất khác biệt rõ rệt (SANPHAM).
- **Phân mảnh ngang dẫn xuất:** Chia quan hệ thành viên dựa trên liên kết với quan hệ chủ đã được phân mảnh. Áp dụng cho các quan hệ phụ thuộc (TONKHO, DONHANG, CHITIETDONHANG).

2.1.1 Phân mảnh ngang cơ sở

Phân mảnh ngang cơ sở được áp dụng cho hai quan hệ độc lập là KHO và KHACHHANG. Cả hai đều chứa thuộc tính KhuVuc làm tiêu chí phân vùng tự nhiên, tuy ý nghĩa nghiệp vụ khác biệt: KHO.KhuVuc biểu diễn vị trí địa lý vật lý của kho hàng, trong khi KHACHHANG.KhuVuc là khu vực đăng ký của khách hàng.

Quá trình phân mảnh được thực hiện qua hai giai đoạn: xác định tập vị từ tối thiểu đầy đủ bằng thuật toán COM_MIN, sau đó sinh các mảnh bằng thuật toán PHORIZONTAL.

Phân mảnh bảng KHO

Bước 1 — Xác định các ứng dụng và tần suất truy xuất. Trước khi xác định vị từ phân mảnh, cần phân tích các ứng dụng (truy vấn) thường xuyên truy cập bảng KHO và tần suất truy cập tại mỗi trạm.

Bảng 2.1: Tần suất truy xuất bảng KHO theo ứng dụng và trạm (lượt/tháng)

Ứng dụng	Mô tả	S1	S2	S3
q_1	Quản lý nhập xuất kho miền Bắc	30	0	0
q_2	Quản lý nhập xuất kho miền Trung	0	30	0
q_3	Quản lý nhập xuất kho miền Nam	0	0	30
q_4	Thống kê tổng quan toàn hệ thống	5	5	5

Bảng 2.1 cho thấy phần lớn truy xuất mang tính cục bộ theo khu vực: q_1 chỉ truy cập kho miền Bắc tại S1, q_2 chỉ truy cập kho miền Trung tại S2, q_3 chỉ truy cập kho miền Nam tại S3. Truy vấn toàn cục q_4 chiếm tỷ lệ nhỏ (khoảng 14% tổng lượt truy cập). Tỷ lệ cục bộ cao này xác nhận tính hợp lý của phân mảnh ngang theo KhuVuc.

Bước 2 — Áp dụng thuật toán COM_MIN. Thuật toán COM_MIN xác định tập vị từ đơn giản đầy đủ và tối thiểu Pr' từ tập vị từ ban đầu Pr .

Xác định tập vị từ đơn giản ban đầu:

Dựa trên phân tích ứng dụng, thuộc tính phân mảnh được chọn là KhuVuc. Tập vị từ đơn giản:

$$Pr = \{p_1, p_2, p_3\}$$

trong đó:

$$p_1 : \text{KhuVuc} = \text{'MB'}$$

$$p_2 : \text{KhuVuc} = \text{'MT'}$$

$$p_3 : \text{KhuVuc} = \text{'MN'}$$

Kiểm tra tính đầy đủ:

Tập Pr là đầy đủ nếu mỗi mảnh sinh ra từ Pr được truy xuất đồng nhất bởi ít nhất một ứng dụng. Ràng buộc CHECK ($\text{KhuVuc IN ('MB', 'MT', 'MN')}$) tại mức lược

đồ đảm bảo miền giá trị của KhuVuc chính xác là {'MB', 'MT', 'MN'}, do đó:

$$p_1 \vee p_2 \vee p_3 \equiv true \quad (\text{phủ kín toàn bộ miền giá trị})$$

Mỗi vị từ p_i tạo ra một tập con bản ghi được ứng dụng q_i truy xuất với tần suất cao (30 lượt/tháng), thỏa mãn tính đầy đủ.

Kiểm tra tính tối thiểu (liên quan):

Vị từ p_i là liên quan nếu tồn tại ứng dụng truy cập các bản ghi thỏa p_i khác biệt so với các bản ghi không thỏa p_i .

- p_1 liên quan: Ứng dụng q_1 truy cập $\sigma_{p_1}(\text{KHO})$ với tần suất 30 lượt/tháng tại S1, nhưng không bao giờ truy cập $\sigma_{\neg p_1}(\text{KHO})$.
- p_2 liên quan: Ứng dụng q_2 truy cập $\sigma_{p_2}(\text{KHO})$ với tần suất 30 lượt/tháng tại S2, nhưng không bao giờ truy cập $\sigma_{\neg p_2}(\text{KHO})$.
- p_3 liên quan: Tương tự, ứng dụng q_3 chỉ truy cập $\sigma_{p_3}(\text{KHO})$ tại S3.

Không có vị từ nào thừa (loại bỏ bất kỳ p_i nào sẽ khiến các bản ghi thuộc khu vực đó bị gộp chung với khu vực khác, vi phạm tính đầy đủ).

Kết quả:

$$Pr' = \{p_1, p_2, p_3\}$$

Bước 3 — Áp dụng thuật toán PHORIZONTAL. Thuật toán PHORIZONTAL sinh các mảnh ngang từ tập vị từ hội sơ cấp (tổ hợp đầy đủ các vị từ trong Pr').

Sinh tập vị từ hội sơ cấp M :

Từ $Pr' = \{p_1, p_2, p_3\}$, tập vị từ hội sơ cấp đầy đủ gồm $2^3 = 8$ tổ hợp:

$$M = \{m_i = p_1^* \wedge p_2^* \wedge p_3^* \mid p_k^* \in \{p_k, \neg p_k\}\}$$

Loại bỏ vị từ hội mâu thuẫn:

Do KhuVuc là thuộc tính đơn trị (mỗi bản ghi chỉ có đúng một giá trị), bất kỳ tổ hợp nào chứa hai vị từ dương đồng thời đều mâu thuẫn. Ví dụ: $p_1 \wedge p_2$ yêu cầu KhuVuc vừa bằng 'MB' vừa bằng 'MT', điều này không thể xảy ra.

Trong 8 tổ hợp, chỉ 3 tổ hợp không mâu thuẫn:

$$m_1 = p_1 \wedge \neg p_2 \wedge \neg p_3 \equiv \text{KhuVuc} = \text{'MB'}$$

$$m_2 = \neg p_1 \wedge p_2 \wedge \neg p_3 \equiv \text{KhuVuc} = \text{'MT'}$$

$$m_3 = \neg p_1 \wedge \neg p_2 \wedge p_3 \equiv \text{KhuVuc} = \text{'MN'}$$

Các tổ hợp còn lại (m_4 đến m_8) đều mâu thuẫn do chứa hai hoặc ba vị từ dương, hoặc phủ định cả ba vị từ (trường hợp $\neg p_1 \wedge \neg p_2 \wedge \neg p_3$ cũng mâu thuẫn vì ràng buộc CHECK buộc KhuVuc phải thuộc một trong ba giá trị).

Kết quả phân mảnh bảng KHO:

$$KHO_1 = \sigma_{m_1}(KHO) = \sigma_{KhuVuc='MB'}(KHO) \rightarrow \text{Cấp phát tại S1} \quad (2.1)$$

$$KHO_2 = \sigma_{m_2}(KHO) = \sigma_{KhuVuc='MT'}(KHO) \rightarrow \text{Cấp phát tại S2} \quad (2.2)$$

$$KHO_3 = \sigma_{m_3}(KHO) = \sigma_{KhuVuc='MN'}(KHO) \rightarrow \text{Cấp phát tại S3} \quad (2.3)$$

Phân mảnh bảng KHACHHANG

Bảng KHACHHANG có cấu trúc phân mảnh tương tự bảng KHO: cùng thuộc tính phân mảnh KhuVuc, cùng miền giá trị {'MB', 'MT', 'MN'}, và cùng đặc điểm truy xuất cục bộ theo khu vực.

Bảng 2.2: Tần suất truy xuất bảng KHACHHANG theo ứng dụng và trạm (lượt/tháng)

Ứng dụng	Mô tả	S1	S2	S3
q_5	Chăm sóc khách hàng miền Bắc	35	0	0
q_6	Xử lý đơn hàng khách miền Trung	0	35	0
q_7	Quản lý khách hàng miền Nam	0	0	35
q_8	Phân tích khách hàng toàn hệ thống	3	3	3

Áp dụng COM_MIN và PHORIZONTAL với cùng tập vị từ $Pr' = \{p_1, p_2, p_3\}$ (xét trên KHACHHANG.KhuVuc), ta được kết quả:

$$KH_1 = \sigma_{KhuVuc='MB'}(KHACHHANG) \rightarrow \text{Cấp phát tại S1} \quad (2.4)$$

$$KH_2 = \sigma_{KhuVuc='MT'}(KHACHHANG) \rightarrow \text{Cấp phát tại S2} \quad (2.5)$$

$$KH_3 = \sigma_{KhuVuc='MN'}(KHACHHANG) \rightarrow \text{Cấp phát tại S3} \quad (2.6)$$

Mặc dù cùng sử dụng thuộc tính KhuVuc, hai phép phân mảnh này độc lập về mặt ngữ nghĩa: phân mảnh KHO nhóm kho hàng theo vị trí vật lý, phân mảnh KHACHHANG nhóm khách hàng theo khu vực đăng ký. Sự phân biệt này sẽ tạo ra thách thức cho bảng CHITIETDONHANG ở mục 2.1.3, khi hai chiều phân mảnh giao nhau.

2.1.2 Phân mảnh dọc bảng SANPHAM

Mở rộng lược đồ

Để phục vụ chiến lược phân mảnh dọc, bảng SANPHAM được bổ sung thuộc tính MoTa lưu trữ mô tả chi tiết sản phẩm (nội dung văn bản dài, có thể kèm thông số kỹ thuật).

Lược đồ mở rộng:

$SANPHAM(\underline{MaSP}, TenSP, GiaBan, MaDM, MoTa)$

Câu lệnh bổ sung:

```
ALTER TABLE SANPHAM ADD MoTa NVARCHAR(MAX);
```

Thuộc tính $MoTa$ khác biệt rõ rệt với các thuộc tính còn lại: dung lượng lớn (kiểu $NVARCHAR(MAX)$), tần suất truy xuất thấp (chỉ khi khách hàng xem chi tiết một sản phẩm cụ thể). Sự khác biệt này là tiền đề cho phân mảnh dọc.

Phân tích tần suất truy xuất thuộc tính

Để xác định cách chia nhóm thuộc tính, ta phân tích ma trận sử dụng thuộc tính bởi các ứng dụng chính.

Bảng 2.3: Ma trận sử dụng thuộc tính bảng $SANPHAM$ (1 = có truy xuất, 0 = không)

Ứng dụng	Mô tả	$MaSP$	$TenSP$	$GiaBan$	$MaDM$	$MoTa$	Tần suất
q_9	Duyệt danh mục sản phẩm	1	1	1	1	0	Cao
q_{10}	Xem chi tiết sản phẩm	1	0	0	0	1	Thấp
q_{11}	Xử lý đơn hàng	1	0	1	0	0	Cao
q_{12}	Báo cáo theo danh mục	1	0	1	1	0	Trung bình

Từ bảng 2.3, ta nhận thấy hai nhóm thuộc tính có mẫu truy xuất khác biệt:

- **Nhóm 1** — $A_1 = \{MaSP, TenSP, GiaBan, MaDM\}$: Được sử dụng bởi q_9, q_{11}, q_{12} với tần suất cao. Đây là các thuộc tính “nhẹ” (dung lượng nhỏ) phục vụ hiển thị danh sách sản phẩm, xử lý đơn hàng, và thống kê.
- **Nhóm 2** — $A_2 = \{MaSP, MoTa\}$: Chỉ được sử dụng bởi q_{10} với tần suất thấp. Thuộc tính $MoTa$ có dung lượng lớn, chỉ cần khi khách bấm xem chi tiết một sản phẩm cụ thể.

Thuộc tính $MaSP$ (khóa chính) xuất hiện ở cả hai nhóm — đây là yêu cầu bắt buộc để đảm bảo tính phục hồi khi cần tái tạo quan hệ ban đầu bằng phép nối tự nhiên.

Kết quả phân mảnh dọc

Áp dụng phép chiếu Π trên hai nhóm thuộc tính:

$$SP_PUBLIC = \Pi_{MaSP, TenSP, GiaBan, MaDM}(SANPHAM) \quad (2.7)$$

$$SP_DETAIL = \Pi_{MaSP, MoTa}(SANPHAM) \quad (2.8)$$

Bảng 2.4: Đặc điểm hai mảnh dọc của bảng SANPHAM

Mảnh	Thuộc tính	Đặc điểm	Chiến lược cấp phát
SP_PUBLIC	MaSP, TenSP, GiaBan, MaDM	Dung lượng nhỏ, truy xuất cao	Nhân bản tại cả 3 trạm
SP_DETAIL	MaSP, MoTa	Dung lượng lớn, truy xuất thấp	Tập trung tại S1

Lợi ích của phân mảnh dọc:

- Khi khách hàng duyệt danh mục (q_9), hệ thống chỉ cần truy xuất SP_PUBLIC — mảnh này được nhân bản tại trạm cục bộ nên chi phí truyền thông bằng không.
- Dữ liệu MoTa dung lượng lớn không bị truyền qua mạng trong các truy vấn thường xuyên, tiết kiệm đáng kể băng thông.
- Khi cần xem chi tiết (q_{10}), truy vấn chỉ truyền MoTa của duy nhất sản phẩm được chọn (thay vì toàn bộ bảng), giảm thiểu lượng dữ liệu truyền tải.

2.1.3 Phân mảnh ngang dẫn xuất

Phân mảnh ngang dẫn xuất áp dụng cho các quan hệ thành viên dựa trên liên kết với quan hệ chủ đã được phân mảnh, thông qua **phép nối nửa** ($\bowtie$). Mục tiêu là đảm bảo tính cục bộ: dữ liệu thành viên nằm cùng trạm với dữ liệu chủ tương ứng, giúp phép nối giữa chúng thực hiện hoàn toàn cục bộ mà không cần truyền dữ liệu liên trạm.

Phân mảnh bảng TONKHO

Bảng TONKHO là thành viên của KHO (qua khóa ngoại MaKho). Mỗi bản ghi tồn kho gắn với đúng một kho, nên phân mảnh dẫn xuất theo KHO đảm bảo tính cục bộ tuyệt đối: dữ liệu tồn kho của kho miền Bắc nằm tại S1, miền Trung tại S2, miền Nam tại S3.

$$TONKHO_1 = TONKHO \bowtie_{MaKho} KHO_1 \quad (2.9)$$

$$TONKHO_2 = TONKHO \bowtie_{MaKho} KHO_2 \quad (2.10)$$

$$TONKHO_3 = TONKHO \bowtie_{MaKho} KHO_3 \quad (2.11)$$

Tương đương với phép chọn:

$$\text{TONKHO}_i = \sigma_{\text{MaKho} \in \Pi_{\text{MaKho}}(\text{KHO}_i)}(\text{TONKHO})$$

Nhờ phân mảnh này, các thao tác cập nhật tồn kho (nhập hàng, xuất hàng) — vốn chiếm phần lớn giao dịch hàng ngày — được xử lý hoàn toàn cục bộ tại trạm quản lý kho.

Phân mảnh bảng DONHANG

Bảng DONHANG là thành viên của KHACHHANG (qua khóa ngoại MaKH). Mỗi đơn hàng thuộc về một khách hàng, và khách hàng đã được phân theo khu vực, nên đơn hàng cũng được phân mảnh theo khu vực khách hàng.

$$\text{DH}_1 = \text{DONHANG} \bowtie_{\text{MaKH}} \text{KH}_1 \quad (2.12)$$

$$\text{DH}_2 = \text{DONHANG} \bowtie_{\text{MaKH}} \text{KH}_2 \quad (2.13)$$

$$\text{DH}_3 = \text{DONHANG} \bowtie_{\text{MaKH}} \text{KH}_3 \quad (2.14)$$

Phân mảnh này đảm bảo: khi khách hàng miền Bắc truy cập lịch sử đơn hàng, hệ thống chỉ cần truy vấn tại S1 mà không cần liên lạc với S2 hay S3.

Phân mảnh bảng CHITIETDONHANG

Bảng CHITIETDONHANG (viết tắt CTDH) là trường hợp phức tạp nhất vì có **hai quan hệ chủ** nằm ở các chiều phân mảnh khác nhau:

- **Quan hệ chủ 1** — DONHANG: Phân mảnh theo khu vực khách hàng. Nếu chọn DONHANG làm mỏ neo, CTDH sẽ nằm cùng trạm với đơn hàng của khách.
- **Quan hệ chủ 2** — KHO (qua MaKhoXuat): Phân mảnh theo khu vực kho. Nếu chọn KHO làm mỏ neo, CTDH sẽ nằm cùng trạm với kho xuất hàng.

Phân tích lựa chọn:

Bảng 2.5: So sánh hai phương án phân mảnh CHITIETDONHANG

Tiêu chí	Mỏ neo DONHANG	Mỏ neo KHO
Truy vấn thường xuyên	Xem lịch sử đơn, tính tổng tiền → cục bộ	Xem lịch sử đơn → liên trạm
Xuất hàng	Cập nhật tồn kho có thể liên trạm	Cập nhật tồn kho → cục bộ
Tần suất truy vấn đọc	Cao (khách hàng xem đơn liên tục)	Thấp hơn
Tần suất cập nhật	Thấp (chỉ khi tạo đơn mới)	Thấp (chỉ khi tạo đơn mới)

Quyết định: Chọn DONHANG làm mỏ neo phân mảnh vì:

- (1) Hầu hết truy vấn liên quan đến chi tiết đơn hàng xuất phát từ phía khách hàng (xem lịch sử đơn, tính tổng tiền, kiểm tra trạng thái). Phân mảnh theo DONHANG giúp phép nối $DH_i \bowtie CTDH_i$ thực hiện hoàn toàn cục bộ.
- (2) Chi phí cập nhật tồn kho liên trạm (khi kho xuất nằm ở trạm khác) là chấp nhận được vì được bảo vệ bởi giao thức cam kết hai pha (trình bày ở chương 4).
- (3) Tỷ lệ đơn hàng cần xuất từ kho khác khu vực là thiểu số — phần lớn đơn hàng được đáp ứng bởi kho cục bộ.

Kết quả phân mảnh:

$$CTDH_1 = CHITIETDONHANG \bowtie_{MaDH} DH_1 \quad (2.15)$$

$$CTDH_2 = CHITIETDONHANG \bowtie_{MaDH} DH_2 \quad (2.16)$$

$$CTDH_3 = CHITIETDONHANG \bowtie_{MaDH} DH_3 \quad (2.17)$$

Ví dụ minh họa: Khách hàng miền Bắc (thuộc KH_1) đặt đơn hàng gồm 10 sản phẩm A. Kho Hà Nội (S1) chỉ có 4, kho Đà Nẵng (S2) có 6. Hệ thống tạo hai bản ghi CTDH: cả hai đều thuộc mảnh $CTDH_1$ (tại S1, cùng trạm với DH_1), dù bản ghi thứ hai có $MaKhoXuat$ trỏ đến kho ở S2. Khi khách xem đơn hàng, phép nối $DH_1 \bowtie CTDH_1$ thực hiện hoàn toàn cục bộ tại S1. Việc trừ tồn kho kho Đà Nẵng cần giao dịch liên trạm nhưng chỉ xảy ra một lần khi tạo đơn.

2.1.4 Kiểm tra tính đúng đắn của phân mảnh

Theo lý thuyết cơ sở dữ liệu phân tán, mọi phép phân mảnh phải thỏa mãn ba quy tắc đúng đắn: tính đầy đủ, tính phục hồi, và tính tách biệt.

Tính đầy đủ

Mỗi bản ghi trong quan hệ toàn cục phải xuất hiện trong ít nhất một mảnh.

Bảng 2.6: Kiểm tra tính đầy đủ

Quan hệ	Chứng minh
KHO	Ràng buộc CHECK buộc KhuVuc $\in \{'MB', 'MT', 'MN'\}$. Ba mảnh phủ kín toàn bộ miền giá trị: $KHO_1 \cup KHO_2 \cup KHO_3 = KHO$.
KHACHHANG	Tương tự KHO, ba mảnh phủ kín miền giá trị KhuVuc.
SANPHAM	Phân mảnh dọc: $A_1 \cup A_2 = \{MaSP, TenSP, GiaBan, MaDM, MoTa\} =$ tập thuộc tính đầy đủ.
TONKHO	Mỗi bản ghi TONKHO có MaKho tham chiếu đến đúng một bản ghi KHO. Vì KHO đã được phân mảnh đầy đủ, mọi TONKHO thuộc đúng một $TONKHO_i$.
DONHANG	Mỗi DONHANG có MaKH tham chiếu đến đúng một KHACHHANG. Vì KHACHHANG đã đầy đủ, mọi DONHANG thuộc đúng một DH_i .
CTDH	Mỗi CTDH có MaDH tham chiếu đến đúng một DONHANG. Vì DONHANG đã đầy đủ, mọi CTDH thuộc đúng một $CTDH_i$.

Tính phục hồi

Quan hệ toàn cục phải tái tạo được từ các mảnh mà không mất dữ liệu.

Phân mảnh ngang — phục hồi bằng phép hợp:

$$KHO = KHO_1 \cup KHO_2 \cup KHO_3$$

$$KHACHHANG = KH_1 \cup KH_2 \cup KH_3$$

$$TONKHO = TONKHO_1 \cup TONKHO_2 \cup TONKHO_3$$

$$DONHANG = DH_1 \cup DH_2 \cup DH_3$$

$$CTDH = CTDH_1 \cup CTDH_2 \cup CTDH_3$$

Phân mảnh dọc — phục hồi bằng phép nối tự nhiên qua khóa chính:

$$SANPHAM = SP_PUBLIC \bowtie_{MaSP} SP_DETAIL$$

Phép nối tự nhiên qua MaSP (khóa chính) đảm bảo không sinh bản ghi giả (spurious tuples) vì mối quan hệ giữa hai mảnh là một-một.

Tính tách biệt

Các mảnh không được chồng lấn (đối với phân mảnh ngang: không có bản ghi trùng; đối với phân mảnh dọc: không có thuộc tính phi khóa trùng).

Phân mảnh ngang:

- Các vị từ phân mảnh (p_1, p_2, p_3) loại trừ nhau do KhuVuc là thuộc tính đơn trị: một bản ghi không thể đồng thời có KhuVuc = 'MB' và KhuVuc = 'MT'.
- Do đó: $KHO_i \cap KHO_j = \emptyset$ với $i \neq j$. Tương tự cho KHACHHANG, TONKHO, DONHANG, CTDH.

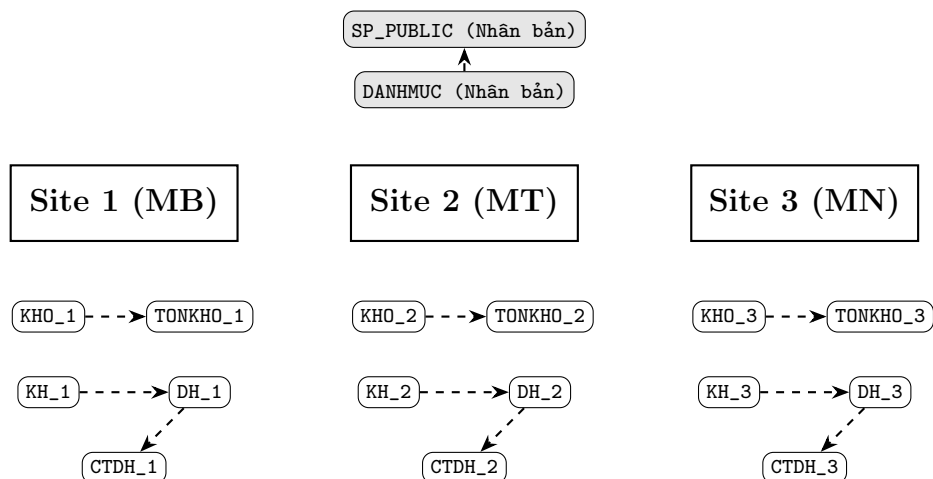
Phân mảnh dọc:

- $A_1 \cap A_2 = \{\text{MaSP}\}$ — chỉ có khóa chính trùng, không có thuộc tính phi khóa nào xuất hiện ở cả hai mảnh. Điều này thỏa mãn tính tách biệt cho phân mảnh dọc.

2.1.5 Tổng hợp kết quả phân mảnh

Bảng 2.7: Tổng hợp toàn bộ kết quả phân mảnh dữ liệu

Quan hệ toàn cục	Kiểu phân mảnh	Các mảnh	Phân bố
DANHMUC	Không phân mảnh	(giữ nguyên)	Nhân bản 3 trạm
SANPHAM	Dọc	SP_PUBLIC, SP_DETAIL	Xem mục 2.2
KHO	Ngang cơ sở	$KHO_{1,2,3}$	S1, S2, S3
KHACHHANG	Ngang cơ sở	$KH_{1,2,3}$	S1, S2, S3
TONKHO	Ngang dẫn xuất	$TONKHO_{1,2,3}$	S1, S2, S3
DONHANG	Ngang dẫn xuất	$DH_{1,2,3}$	S1, S2, S3
CTDH	Ngang dẫn xuất	$CTDH_{1,2,3}$	S1, S2, S3



Hình 2.1: Sơ đồ tổng hợp phân mảnh dữ liệu ngang dẫn xuất tại các trạm

2.2 Chiến lược cấp phát và nhân bản

Chiến lược cấp phát xác định vị trí lưu trữ vật lý của từng mảnh dữ liệu trên các trạm. Chiến lược nhân bản quyết định dữ liệu nào cần sao chép tại nhiều trạm. Cả hai quyết định dựa trên nguyên tắc tối ưu hàm chi phí: cân bằng giữa chi phí lưu trữ (tăng khi nhân bản) và chi phí truyền thông (giảm khi nhân bản).

2.2.1 Biện luận chiến lược nhân bản

Nguyên tắc quyết định: Nếu tỷ lệ truy vấn chỉ đọc lớn hơn nhiều so với truy vấn cập nhật ($R \gg U$), thì nhân bản có lợi. Ngược lại, nếu dữ liệu bị cập nhật thường xuyên, nhân bản gây thất cổ chai mạng do phải đồng bộ mọi thay đổi giữa các bản sao.

Bảng 2.8: Phân tích tỷ lệ đọc/ghi để quyết định nhân bản

Dữ liệu	Đọc (R)	Ghi (U)	Tỷ lệ R/U	Quyết định
DANHMUC	Rất cao	Rất thấp	$\gg 1$	Nhân bản đầy đủ
SP_PUBLIC	Rất cao	Thấp	$\gg 1$	Nhân bản đầy đủ
SP_DETAIL	Thấp	Thấp	≈ 1	Tập trung tại S1
TONKHO	Trung bình	Cao	< 1	Không nhân bản
KHO	Trung bình	Thấp	> 1	Không nhân bản
DONHANG	Trung bình	Trung bình	≈ 1	Không nhân bản
CTDH	Trung bình	Trung bình	≈ 1	Không nhân bản

Biện luận chi tiết cho từng quyết định:

- (1) **DANHMUC — nhân bản đầy đủ:** Danh mục sản phẩm là dữ liệu tham chiếu tĩnh, hiếm khi thay đổi (thêm danh mục mới là sự kiện hiếm). Mọi trạm đều cần danh mục để hiển thị giao diện web và phân loại sản phẩm. Nhân bản giúp chi phí truyền thông cho các truy vấn hiển thị danh mục bằng không. Chi phí đồng bộ khi cập nhật danh mục là tối thiểu do tần suất ghi rất thấp.
- (2) **SP_PUBLIC — nhân bản đầy đủ:** Mảnh chứa thông tin cơ bản sản phẩm (tên, giá, danh mục) được truy xuất liên tục bởi mọi trạm khi khách duyệt sản phẩm. Dung lượng nhỏ nên chi phí lưu trữ tại 3 trạm không đáng kể. Khi giá sản phẩm thay đổi, chi phí đồng bộ chấp nhận được vì tần suất điều chỉnh giá thấp hơn nhiều so với tần suất đọc.
- (3) **SP_DETAIL — tập trung tại S1:** Mảnh chứa mô tả chi tiết có dung lượng lớn (NVARCHAR(MAX)) nhưng tần suất truy xuất thấp. Nhân bản tạo lãng phí lưu trữ. Khi S2 hoặc S3 cần MoTa, truy vấn truyền qua mạng về S1 — chi phí này chấp nhận được vì chỉ truyền mô tả của một sản phẩm cụ thể (không phải toàn bộ).

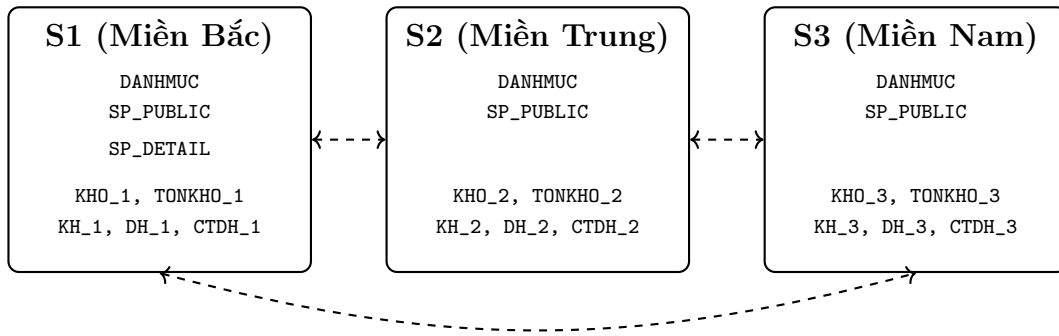
- (4) Các bảng giao dịch (TONKHO, KHO, DONHANG, CTDH) — **không nhân bản**: Đây là dữ liệu biến động. Đặc biệt TONKHO bị cập nhật sau mỗi lần xuất nhập hàng. Nhân bản sẽ buộc hệ thống phải gửi thông điệp cập nhật và chờ xác nhận từ mọi bản sao sau mỗi giao dịch, tạo thất cổ chai nghiêm trọng trên mạng.

2.2.2 Sơ đồ cấp phát dữ liệu phân tán

Bảng 2.9 tổng hợp vị trí lưu trữ vật lý của mọi mảnh dữ liệu.

Bảng 2.9: Sơ đồ cấp phát dữ liệu tại các trạm

Mảnh dữ liệu	S1 (MB)	S2 (MT)	S3 (MN)	Chiến lược
DANHMUC	✓	✓	✓	Nhân bản đầy đủ
SP_PUBLIC	✓	✓	✓	Nhân bản đầy đủ
SP_DETAIL	✓			Tập trung tại S1
KHO_1	✓			Phân mảnh không nhân bản
KHO_2		✓		
KHO_3			✓	
TONKHO_1	✓			Phân mảnh không nhân bản
TONKHO_2		✓		
TONKHO_3			✓	
KH_1	✓			Phân mảnh không nhân bản
KH_2		✓		
KH_3			✓	
DH_1	✓			Phân mảnh không nhân bản
DH_2		✓		
DH_3			✓	
CTDH_1	✓			Phân mảnh không nhân bản
CTDH_2		✓		
CTDH_3			✓	



Hình 2.2: Sơ đồ cấp phát dữ liệu vật lý trên các máy chủ

Tổng kết cấp phát:

- Mỗi trạm lưu trữ: 2 bảng nhân bản (DANH MỤC, SP_PUBLIC) + 5 mảnh cục bộ (KHO_i , KH_i , $TONKHO_i$, DH_i , $CTDH_i$).
- S1 lưu thêm SP_DETAIL (server trung tâm cho dữ liệu mô tả sản phẩm).
- Tổng dữ liệu nhân bản: 2 bảng $\times$ 3 trạm = 6 bản sao (so với 2 bản gốc). Chi phí lưu trữ tăng 200% cho hai bảng nhỏ — đánh đổi xứng đáng để loại bỏ hoàn toàn truyền thông cho truy vấn danh mục và sản phẩm.

Tổng kết chương 2

Chương này đã hoàn thành thiết kế cơ sở dữ liệu phân tán qua hai bước chính:

1. Phân mảnh dữ liệu:

- Phân mảnh ngang cơ sở bảng KHO và KHACHHANG bằng thuật toán COM_MIN và PHORIZONTAL, mỗi bảng chia thành 3 mảnh theo khu vực.
- Phân mảnh dọc bảng SANPHAM thành SP_PUBLIC (thuộc tính truy xuất cao) và SP_DETAIL (thuộc tính dung lượng lớn, truy xuất thấp).
- Phân mảnh ngang dẫn xuất 3 bảng thành viên (TONKHO, DONHANG, CHITIETDONHANG) bằng phép nối nửa với quan hệ chủ.
- Kiểm chứng tính đúng đắn: đầy đủ, phục hồi, tách biệt cho mọi phép phân mảnh.

2. Cấp phát và nhân bản:

- Nhân bản đầy đủ cho bảng tham chiếu (DANH MỤC, SP_PUBLIC).
- Lưu tập trung SP_DETAIL tại S1.
- Phân mảnh không nhân bản cho dữ liệu giao dịch, tối ưu cho cập nhật cục bộ.

Kết quả thiết kế đảm bảo nguyên tắc cục bộ hóa: phần lớn nghiệp vụ thường ngày (quản lý kho, xử lý đơn hàng, xem lịch sử) được thực hiện hoàn toàn tại trạm cục bộ. Các truy vấn liên trạm (thống kê toàn hệ thống, đặt hàng liên kho) sẽ được tối ưu hóa ở chương 3 bằng kỹ thuật xử lý truy vấn phân tán.

Chương 3

Tối ưu hóa truy vấn phân tán

Chương này trình bày quá trình xử lý và tối ưu hóa năm truy vấn nghiệp vụ tiêu biểu trong hệ thống bán hàng đa kho. Mỗi truy vấn được phân tích nghiêm ngặt qua bốn bước của quy trình xử lý truy vấn phân tán, nhằm tối thiểu hóa chi phí truyền thông liên trạm — yếu tố quyết định hiệu năng của toàn hệ thống.

3.1 Tổng quan xử lý truy vấn phân tán

Trong hệ quản trị cơ sở dữ liệu phân tán, một câu truy vấn SQL được viết trên lược đồ toàn cục phải trải qua bốn bước xử lý tuần tự trước khi thực thi:

Bước 1: Phân rã truy vấn. Câu lệnh SQL được chuyển đổi thành biểu thức đại số quan hệ (cây đại số quan hệ) trên lược đồ toàn cục. Tại bước này, truy vấn chưa biết dữ liệu nằm ở trạm nào.

Bước 2: Cục bộ hóa dữ liệu. Các quan hệ toàn cục trong biểu thức được thay thế bằng các mảnh vật lý tương ứng. Sau đó, áp dụng **quy tắc rút gọn** để loại bỏ các mảnh rỗng ($\emptyset$) do mâu thuẫn vị từ, thu gọn cây truy vấn.

Bước 3: Tối ưu hóa toàn cục. Xác định chiến lược thực thi tối ưu xuyên trạm: sắp xếp thứ tự phép nối, sử dụng phép nối nửa ($\bowtie$) để giảm lượng dữ liệu truyền qua mạng, và đẩy các phép toán tổng hợp xuống cục bộ.

Bước 4: Tối ưu hóa cục bộ. Từng trạm tự chọn đường dẫn truy xuất vật lý tốt nhất (sử dụng chỉ mục, quét tuần tự, v.v.) cho phần truy vấn được giao. Bước này do hệ quản trị cục bộ tự xử lý.

3.1.1 Các quy tắc rút gọn phân mảnh

Quy tắc rút gọn là công cụ cốt lõi ở bước cục bộ hóa, giúp loại bỏ các mảnh không liên quan và thu hẹp phạm vi tìm kiếm.

Bảng 3.1: Các quy tắc rút gọn áp dụng trong hệ thống

Quy tắc	Nội dung
Rút gọn phân mảnh ngang cơ sở	Nếu $R_i = \sigma_{p_i}(R)$ và truy vấn chứa $\sigma_{p_j}(R)$ với $p_i \wedge p_j \equiv \text{false}$, thì $\sigma_{p_j}(R_i) = \emptyset$, mảnh R_i bị loại bỏ. Ví dụ: $\sigma_{\text{KhuVuc}=\text{'MB'}}(\text{KHO}_2) = \emptyset$ vì KHO_2 chỉ chứa bản ghi có $\text{KhuVuc} = \text{'MT'}$.
Rút gọn phân mảnh ngang dẫn xuất	Nếu $S_i = S \bowtie R_i$ và mảnh R_i đã bị loại bỏ, thì S_i cũng bị loại bỏ theo (vì S_i chỉ chứa bản ghi nối được với R_i). Ví dụ: Nếu KHO_2 bị loại, thì TONKHO_2 (dẫn xuất theo KHO_2) cũng bị loại.
Rút gọn phân mảnh dọc	Nếu truy vấn chỉ dùng các thuộc tính thuộc mảnh R_k , thì các mảnh dọc khác bị loại bỏ (không cần nối tự nhiên để phục hồi). Ví dụ: Truy vấn chỉ cần TenSP , $\text{GiaBan} \rightarrow$ chỉ dùng SP_PUBLIC , loại SP_DETAIL .

3.1.2 Các kỹ thuật tối ưu hóa toàn cục

Hai kỹ thuật tối ưu chính được áp dụng trong chương này:

- **Phép nối nửa:** Thay vì truyền toàn bộ bảng R từ trạm A sang trạm B để thực hiện $R \bowtie S$, ta chỉ truyền tập khóa nối $\pi_{\text{key}}(R)$. Trạm B dùng tập khóa này để lọc S trước, giảm đáng kể lượng dữ liệu truyền qua mạng.
- **Tổng hợp cục bộ:** Thay vì kéo toàn bộ dữ liệu thô về một trạm để thực hiện GROUP BY, ta đẩy phép tổng hợp xuống từng trạm. Mỗi trạm tính kết quả cục bộ (vài dòng) rồi gửi về trạm điều phối để tổng hợp lần cuối, tiết kiệm băng thông đáng kể so với truyền hàng ngàn dòng chi tiết.

3.2 Phân tích và tối ưu 5 truy vấn nghiệp vụ

Nằm truy vấn sau đây bao quát các tình huống phân tán tiêu biểu: truy vấn cục bộ hoàn toàn, tổng hợp toàn cục, nối liên trạm với phép nối nửa, và phân tích ảnh hưởng của nhân bản.

3.2.1 Truy vấn 1: Kiểm tra sản phẩm còn hàng tại kho miền Bắc

a) **Mô tả nghiệp vụ và SQL toàn cục.** Khách hàng miền Bắc tra cứu danh sách sản phẩm hiện còn hàng tại các kho trong khu vực, kèm số lượng tồn kho.

```
SELECT SP.TenSP, TK.SoLuongTon
FROM SANPHAM SP
```

```

3 JOIN TONKHO TK ON SP.MaSP = TK.MaSP
4 JOIN KHO K ON TK.MaKho = K.MaKho
5 WHERE K.KhuVuc = 'MB';

```

Đoạn mã 3.1: Truy vấn 1 — SQL toàn cục

b) **Cây đại số quan hệ ban đầu.** Biểu thức đại số quan hệ trên lược đồ toàn cục:

$$\pi_{\text{TenSP}, \text{SoLuongTon}} \left(\text{SANPHAM} \bowtie_{\text{MaSP}} \text{TONKHO} \bowtie_{\text{MaKho}} \sigma_{\text{KhuVuc}='MB'}(\text{KHO}) \right)$$

c) **Cục bộ hóa và thu gọn cây.** Bước c.1 — Thay quan hệ bằng mảnh:

- $\text{KHO} \rightarrow \text{KHO}_1 \cup \text{KHO}_2 \cup \text{KHO}_3$
- $\text{TONKHO} \rightarrow \text{TONKHO}_1 \cup \text{TONKHO}_2 \cup \text{TONKHO}_3$
- $\text{SANPHAM} \rightarrow \text{SP_PUBLIC}$ (mảnh dọc chứa đủ TenSP , nhân bản tại mọi trạm)

Bước c.2 — Áp dụng quy tắc rút gọn:

- *Rút gọn phân mảnh ngang cơ sở:* Phép chọn $\sigma_{\text{KhuVuc}='MB'}$ loại bỏ KHO_2 và KHO_3 do mâu thuẫn vị trí:

$$\sigma_{\text{KhuVuc}='MB'}(\text{KHO}_2) = \emptyset, \quad \sigma_{\text{KhuVuc}='MB'}(\text{KHO}_3) = \emptyset$$

Chỉ giữ lại KHO_1 .

- *Rút gọn phân mảnh dẫn xuất:* TONKHO_i là dẫn xuất của KHO_i . Vì $\text{KHO}_2, \text{KHO}_3$ đã bị loại, $\text{TONKHO}_2, \text{TONKHO}_3$ cũng bị loại theo. Chỉ giữ lại TONKHO_1 .
- *Rút gọn phân mảnh dọc:* Truy vấn chỉ cần TenSP (thuộc SP_PUBLIC), không cần MoTa (thuộc SP_DETAIL). Loại bỏ SP_DETAIL . Lấy bản nhân bản SP_PUBLIC tại S1.

Cây thu gọn:

$$\pi_{\text{TenSP}, \text{SoLuongTon}} \left(\text{SP_PUBLIC}_1 \bowtie_{\text{MaSP}} \text{TONKHO}_1 \bowtie_{\text{MaKho}} \text{KHO}_1 \right)$$

Toàn bộ phép toán giờ chỉ liên quan đến dữ liệu tại **Site 1**.

d) **Tối ưu hóa toàn cục.** Sau khi thu gọn, cây truy vấn đã được **cục bộ hóa hoàn toàn tại Site 1**. Mọi mảnh tham gia ($\text{SP_PUBLIC}_1, \text{TONKHO}_1, \text{KHO}_1$) đều nằm tại cùng một trạm.

Chi phí truyền thông = 0 (Không phát sinh truyền dữ liệu liên trạm)

Đây là kết quả trực tiếp của chiến lược thiết kế ở chương 2: phân mảnh theo khu vực kết hợp nhân bản bảng tham chiếu đảm bảo các truy vấn cục bộ theo vùng không cần liên lạc với trạm khác.

```

1  -- Thực thi hoàn toàn cục bộ tại Site 1 (miền Bắc)
2  SELECT SP.TenSP, TK.SoLuongTon
3  FROM SP_PUBLIC SP
4  JOIN TONKHO_MB TK ON SP.MaSP = TK.MaSP
5  JOIN KHO_MB K ON TK.MaKho = K.MaKho;

```

Đoạn mã 3.2: Truy vấn 1 — SQL thực thi tại Site 1

3.2.2 Truy vấn 2: Tính tổng tồn kho của một sản phẩm trên toàn hệ thống

a) **Mô tả nghiệp vụ và SQL toàn cục.** Bộ phận mua hàng cần kiểm tra tổng tồn kho của sản phẩm mã 'SP001' trên tất cả các kho để quyết định có cần nhập thêm hàng hay không.

```

1  SELECT SUM(SoLuongTon) AS TongTonKho
2  FROM TONKHO
3  WHERE MaSP = 'SP001';

```

Đoạn mã 3.3: Truy vấn 2 — SQL toàn cục

b) **Cây đại số quan hệ ban đầu.**

$$\gamma_{SUM(SoLuongTon)}(\sigma_{MaSP='SP001'}(TONKHO))$$

c) **Cục bộ hóa và thu gọn cây.** Truy vấn không chứa điều kiện về KhuVuc nên không có mảnh nào bị loại bỏ. Phải duyệt toàn bộ ba mảnh:

$$\gamma_{SUM(SoLuongTon)}(\sigma_{MaSP='SP001'}(TONKHO_1 \cup TONKHO_2 \cup TONKHO_3))$$

Đẩy phép chọn xuống từng mảnh:

$$\gamma_{SUM}(\sigma_{MaSP='SP001'}(TONKHO_1) \cup \sigma_{MaSP='SP001'}(TONKHO_2) \cup \sigma_{MaSP='SP001'}(TONKHO_3))$$

d) **Tối ưu hóa toàn cục — tổng hợp cục bộ.** Phương án không tối ưu: Kéo toàn bộ bản ghi $\sigma_{MaSP='SP001'}(TONKHO_i)$ từ ba trạm về một trạm điều phối, sau đó tính SUM. Lượng dữ liệu truyền: 3 bản ghi (mỗi bản ghi gồm MaKho, MaSP, SoLuongTon).

Phương án tối ưu — tổng hợp cục bộ: Đẩy phép SUM xuống từng trạm. Mỗi trạm tính tổng cục bộ, chỉ gửi một giá trị số duy nhất về trạm điều phối:

$$S_1 = \gamma_{SUM(\text{SoLuongTon})}(\sigma_{\text{MaSP}='SP001'}(\text{TONKHO}_1)) \quad \text{— tại } S1$$

$$S_2 = \gamma_{SUM(\text{SoLuongTon})}(\sigma_{\text{MaSP}='SP001'}(\text{TONKHO}_2)) \quad \text{— tại } S2$$

$$S_3 = \gamma_{SUM(\text{SoLuongTon})}(\sigma_{\text{MaSP}='SP001'}(\text{TONKHO}_3)) \quad \text{— tại } S3$$

Kết quả cuối cùng: $\text{TongTonKho} = S_1 + S_2 + S_3$.

Chi phí truyền thông = 3 giá trị số (mỗi trạm gửi 1 số về trạm điều phối)

Trong trường hợp này, do mỗi trạm chỉ có tối đa một bản ghi cho sản phẩm 'SP001', mức tiết kiệm chưa lớn. Tuy nhiên, kỹ thuật tổng hợp cục bộ sẽ phát huy rõ rệt ở truy vấn 4 và 5 khi số lượng bản ghi tham gia tính tổng lên đến hàng ngàn.

```

1 SELECT SUM(TonCucBo) AS TongTonKhoToanHeThong
2 FROM (
3     -- S1: Tổng hợp cục bộ tại miền Bắc
4     SELECT SUM(SoLuongTon) AS TonCucBo
5     FROM TONKHO_MB WHERE MaSP = 'SP001'
6     UNION ALL
7     -- S2: Tổng hợp cục bộ tại miền Trung
8     SELECT SUM(SoLuongTon) AS TonCucBo
9     FROM TONKHO_MT WHERE MaSP = 'SP001'
10    UNION ALL
11    -- S3: Tổng hợp cục bộ tại miền Nam
12    SELECT SUM(SoLuongTon) AS TonCucBo
13    FROM TONKHO_MN WHERE MaSP = 'SP001'
14 ) AS KetQuaCucBo;
```

Đoạn mã 3.4: Truy vấn 2 — SQL thực thi với tổng hợp cục bộ

3.2.3 Truy vấn 3: Tìm đơn hàng giao chéo vùng

a) Mô tả nghiệp vụ và SQL toàn cục. Bộ phận logistics cần tìm các đơn hàng của khách miền Nam có sản phẩm phải xuất từ kho miền Bắc (giao chéo vùng). Đây là truy vấn đo lường hiệu quả phân bổ hàng hóa giữa các khu vực.

```

1 SELECT DISTINCT D.MaDH
2 FROM DONHANG D
3 JOIN CHITIETDONHANG C ON D.MaDH = C.MaDH
4 JOIN KHACHHANG KH ON D.MaKH = KH.MaKH
5 JOIN KHO K ON C.MaKhoXuat = K.MaKho
```

WHERE KH.KhuVuc = 'MN' AND K.KhuVuc = 'MB';

Đoạn mã 3.5: Truy vấn 3 — SQL toàn cục

b) Cây đại số quan hệ ban đầu.

$$\pi_{\text{MaDH}} \left(\sigma_{\text{KhuVuc}='MN'}(\text{KH}) \bowtie \text{D} \bowtie \text{CTDH} \bowtie \sigma_{\text{KhuVuc}='MB'}(\text{KHO}) \right)$$

c) Cục bộ hóa và thu gọn cây. Áp dụng tuần tự các quy tắc rút gọn:

(i) *Rút gọn phân mảnh ngang cơ sở cho KHACHHANG:*

$$\sigma_{\text{KhuVuc}='MN'}(\text{KH}_1 \cup \text{KH}_2 \cup \text{KH}_3) \implies \text{Loại KH}_1, \text{KH}_2. \text{Giữ KH}_3 \text{ (S3)}.$$

(ii) *Rút gọn phân mảnh dẫn xuất cho DONHANG:*

$$\text{DH}_i = \text{DONHANG} \bowtie \text{KH}_i. \text{Vì KH}_1, \text{KH}_2 \text{ đã bị loại} \implies \text{Loại DH}_1, \text{DH}_2. \text{Giữ DH}_3 \text{ (S3)}.$$

(iii) *Rút gọn phân mảnh dẫn xuất cho CHITIETDONHANG:*

$$\text{CTDH}_i = \text{CTDH} \bowtie \text{DH}_i. \text{Vì DH}_1, \text{DH}_2 \text{ đã bị loại} \implies \text{Loại CTDH}_1, \text{CTDH}_2. \text{Giữ CTDH}_3 \text{ (S3)}.$$

(iv) *Rút gọn phân mảnh ngang cơ sở cho KHO:*

$$\sigma_{\text{KhuVuc}='MB'}(\text{KHO}_1 \cup \text{KHO}_2 \cup \text{KHO}_3) \implies \text{Loại KHO}_2, \text{KHO}_3. \text{Giữ KHO}_1 \text{ (S1)}.$$

Cây thu gọn:

$$\pi_{\text{MaDH}} \left(\underbrace{\text{KH}_3 \bowtie \text{DH}_3 \bowtie \text{CTDH}_3}_{\text{Tại S3}} \bowtie \underbrace{\text{KHO}_1}_{\text{Tại S1}} \right)$$

Nhận xét quan trọng: Chuỗi rút gọn dẫn xuất $\text{KH}_3 \rightarrow \text{DH}_3 \rightarrow \text{CTDH}_3$ loại bỏ hoàn toàn nhu cầu nối với bảng KHACHHANG trong câu lệnh thực thi. Điều kiện KhuVuc='MN' đã được “nhúng” vào việc chọn mảnh DH₃ (chỉ chứa đơn hàng của khách miền Nam). Tuy nhiên, phép nối giữa CTDH₃ (S3) và KHO₁ (S1) vẫn là **phép nối liên trạm**, cần tối ưu ở bước tiếp theo.

d) **Tối ưu hóa toàn cục — phép nối nữa.** Để thực hiện phép nối giữa CTDH₃ (tại S3) và KHO₁ (tại S1), thay vì truyền toàn bộ bảng KHO₁ qua mạng, ta áp dụng phép nối nữa.

Quy trình thực hiện:

1. **S1 gửi tập khóa:** Trạm S1 chiếu tập khóa nối $\pi_{\text{MaKho}}(\text{KHO}_1)$ và gửi sang S3. Dữ liệu truyền chỉ gồm danh sách mã kho miền Bắc (vài giá trị VARCHAR(10)).

2. **S3 lọc cục bộ:** Trạm S3 dùng tập khóa nhận được để lọc $CTDH_3$:

$$CTDH'_3 = CTDH_3 \bowtie_{\text{MaKhoXuat}} KHO_1 = \sigma_{\text{MaKhoXuat} \in \pi_{\text{MaKho}}(KHO_1)}(CTDH_3)$$

3. **S3 tính kết quả:** Từ $CTDH'_3$ (đã lọc), S3 nối với DH_3 cục bộ và chiếu π_{MaDH} . Kết quả cuối cùng được tính hoàn toàn tại S3.

Phân tích chi phí truyền thông:

Bảng 3.2: So sánh chi phí truyền thông truy vấn 3

Tiêu chí	Không dùng phép nối nửa	Dùng phép nối nửa
Dữ liệu truyền	Toàn bộ KHO_1 hoặc $CTDH_3$	Chỉ $\pi_{\text{MaKho}}(KHO_1)$
Kích thước ước tính	Hàng trăm KB (toàn bộ bảng)	Vài chục byte (danh sách mã kho)
Số lượt truyền	1 lượt (truyền bảng lớn)	1 lượt (truyền tập khóa nhỏ)

Bất đẳng thức tối ưu được thỏa mãn:

$$|\pi_{\text{MaKho}}(KHO_1)| \ll |KHO_1| \ll |CTDH_3|$$

Bên cạnh phép nối nửa, nhờ đặc tính của phân mảnh ngang dẫn xuất, mảnh DH_3 đã hàm chứa sẵn điều kiện lọc khách hàng miền Nam. Do đó, phép nối với bảng $KHACHHANG$ được loại bỏ hoàn toàn khỏi kế hoạch thực thi, giúp giảm thêm chi phí đọc đĩa và xử lý tại trạm cục bộ. Truy vấn 3 minh họa sự kết hợp hiệu quả giữa hai kỹ thuật tối ưu: *rút gọn dẫn xuất* loại bỏ phép nối thừa, và *phép nối nửa* giảm chi phí truyền thông cho phép nối liên trạm còn lại.

```

1  -- Thực thi tại S3 (miền Nam)
2  SELECT DISTINCT D.MaDH
3  FROM DONHANG_MN D
4  JOIN CTDH_MN C ON D.MaDH = C.MaDH
5  WHERE C.MaKhoXuat IN (
6      -- Phép nối nửa: truyền tập khóa từ S1
7      SELECT MaKho FROM KHO_MB
8  );

```

Đoạn mã 3.6: Truy vấn 3 — SQL thực thi với phép nối nửa

3.2.4 Truy vấn 4: Thống kê doanh thu theo tháng của từng kho

a) **Mô tả nghiệp vụ và SQL toàn cục.** Kế toán tổng hợp doanh thu theo từng kho xuất hàng trong tháng hiện tại để đánh giá hiệu quả kinh doanh từng khu vực.

```
1 SELECT C.MaKhoXuat, SUM(C.SoLuong * C.DonGia) AS DoanhThu
2 FROM CHITIETDONHANG C
3 JOIN DONHANG D ON C.MaDH = D.MaDH
4 WHERE MONTH(D.NgayLap) = MONTH(GETDATE())
5      AND YEAR(D.NgayLap) = YEAR(GETDATE())
6 GROUP BY C.MaKhoXuat;
```

Đoạn mã 3.7: Truy vấn 4 — SQL toàn cục

b) **Cây đại số quan hệ ban đầu.**

$$\gamma_{\text{MaKhoXuat}, \text{SUM}(\text{SoLuong} \times \text{DonGia})} \left(\text{CTDH} \bowtie_{\text{MaDH}} \sigma_{\text{thang_hien_tai}}(\text{DONHANG}) \right)$$

c) **Cục bộ hóa và thu gọn cây.** Truy vấn không chứa điều kiện theo khu vực nên phải duyệt toàn bộ ba mảnh của cả CTDH và DONHANG. Tuy nhiên, nhờ đặc tính phân mảnh dẫn xuất, CTDH_i chỉ nối được với DH_i (cùng phân mảnh theo khách hàng):

$$(\text{CTDH}_1 \bowtie \sigma_{\text{thang}}(\text{DH}_1)) \cup (\text{CTDH}_2 \bowtie \sigma_{\text{thang}}(\text{DH}_2)) \cup (\text{CTDH}_3 \bowtie \sigma_{\text{thang}}(\text{DH}_3))$$

Nhận xét quan trọng: Phân mảnh dẫn xuất loại bỏ hoàn toàn các phép nối chéo mảnh. CTDH₁ không bao giờ nối với DH₂ hay DH₃ vì chúng không có bản ghi chung. Điều này biến một phép nối phân tán tiềm năng ($3 \times 3 = 9$ tổ hợp) thành **3 phép nối cục bộ** song song.

d) **Tối ưu hóa toàn cục — tổng hợp cục bộ.** Thay vì truyền toàn bộ dữ liệu chi tiết đơn hàng (hàng ngàn dòng CTDH) về trạm điều phối rồi mới thực hiện GROUP BY, ta đẩy phép tổng hợp xuống từng trạm.

Giai đoạn 1 — Tổng hợp cục bộ (song song tại 3 trạm):

Mỗi trạm i thực hiện phép nối CTDH _{i} $\bowtie$ $\sigma_{\text{thang}}(\text{DH}_i)$ và gom nhóm theo MaKhoXuat:

$$R_i = \gamma_{\text{MaKhoXuat}, \text{SUM}(\text{SoLuong} \times \text{DonGia})} (\text{CTDH}_i \bowtie \sigma_{\text{thang}}(\text{DH}_i))$$

Mỗi R_i chỉ chứa vài dòng (một dòng cho mỗi kho xuất hàng liên quan).

Lưu ý: Do CTDH phân mảnh theo đơn hàng (không phải theo kho xuất), R_1 có thể chứa doanh thu của kho miền Trung hoặc miền Nam (trường hợp khách miền Bắc đặt hàng xuất từ kho khác). Đây là lý do cần giai đoạn 2 để hợp nhất.

Giai đoạn 2 — Tổng hợp toàn cục (tại trạm điều phối):

Trạm điều phối nhận R_1, R_2, R_3 (mỗi tập vài dòng) và thực hiện gom nhóm lần cuối:

$$\text{Kết quả} = \gamma_{\text{MaKhoXuat}, \text{SUM}(\text{DoanhThu})}(R_1 \cup R_2 \cup R_3)$$

Bảng 3.3: So sánh chi phí truyền thông truy vấn 4

Phương án	Dữ liệu truyền qua mạng	Ước lượng kích thước
Không tối ưu (truyền dữ liệu thô)	Toàn bộ CTDH	Hàng ngàn dòng
Tổng hợp cục bộ	$R_1 \cup R_2 \cup R_3$	Khoảng $3k$ dòng (k = số kho)

Với k kho trên toàn hệ thống (giả sử $k = 9$, mỗi miền 3 kho), lượng dữ liệu truyền giảm từ hàng ngàn dòng chi tiết xuống tối đa $3 \times 9 = 27$ dòng kết quả cục bộ.

```
1 SELECT MaKhoXuat, SUM(DoanhThuCucBo) AS DoanhThu
2 FROM (
3     -- Giai đoạn 1: Tổng hợp cục bộ tại S1
4     SELECT C.MaKhoXuat,
5            SUM(C.SoLuong * C.DonGia) AS DoanhThuCucBo
6     FROM CTDH_MB C
7     JOIN DONHANG_MB D ON C.MaDH = D.MaDH
8     WHERE MONTH(D.NgayLap) = MONTH(GETDATE())
9           AND YEAR(D.NgayLap) = YEAR(GETDATE())
10    GROUP BY C.MaKhoXuat
11    UNION ALL
12    -- Giai đoạn 1: Tổng hợp cục bộ tại S2
13    SELECT C.MaKhoXuat,
14           SUM(C.SoLuong * C.DonGia) AS DoanhThuCucBo
15    FROM CTDH_MT C
16    JOIN DONHANG_MT D ON C.MaDH = D.MaDH
17    WHERE MONTH(D.NgayLap) = MONTH(GETDATE())
18           AND YEAR(D.NgayLap) = YEAR(GETDATE())
19    GROUP BY C.MaKhoXuat
20    UNION ALL
21    -- Giai đoạn 1: Tổng hợp cục bộ tại S3
22    SELECT C.MaKhoXuat,
23           SUM(C.SoLuong * C.DonGia) AS DoanhThuCucBo
24    FROM CTDH_MN C
25    JOIN DONHANG_MN D ON C.MaDH = D.MaDH
26    WHERE MONTH(D.NgayLap) = MONTH(GETDATE())
27           AND YEAR(D.NgayLap) = YEAR(GETDATE())
28    GROUP BY C.MaKhoXuat
29 ) AS TongHop
30 -- Giai đoạn 2: Gom nhóm lần cuối theo MaKhoXuat
```

```
3 | GROUP BY MaKhoXuat ;
```

Đoạn mã 3.8: Truy vấn 4 — SQL thực thi với tổng hợp cục bộ hai giai đoạn

3.2.5 Truy vấn 5: Tìm sản phẩm bán chạy nhất toàn hệ thống

a) **Mô tả nghiệp vụ và SQL toàn cục.** Ban giám đốc cần biết sản phẩm nào bán chạy nhất trên toàn quốc để lập kế hoạch nhập hàng và marketing.

```
1 | SELECT SP.TenSP, SUM(C.SoLuong) AS TongBan
2 | FROM SANPHAM SP
3 | JOIN CHITIETDONHANG C ON SP.MaSP = C.MaSP
4 | GROUP BY SP.TenSP
5 | ORDER BY TongBan DESC ;
```

Đoạn mã 3.9: Truy vấn 5 — SQL toàn cục

b) **Cây đại số quan hệ ban đầu.**

$$\tau_{DESC(TongBan)} \left(\gamma_{TenSP, SUM(SoLuong)} (SANPHAM \bowtie_{MaSP} CTDH) \right)$$

c) **Cục bộ hóa và thu gọn cây.**

- CTDH không có điều kiện lọc theo khu vực $\rightarrow$ duyệt cả ba mảnh: $CTDH_1 \cup CTDH_2 \cup CTDH_3$.
- SANPHAM: Truy vấn chỉ cần TenSP (thuộc SP_PUBLIC), loại bỏ SP_DETAIL. Sử dụng bản nhân bản SP_PUBLIC tại mỗi trạm.

Cây sau cục bộ hóa:

$$\tau_{DESC} \left(\gamma_{TenSP, SUM(SoLuong)} (SP_PUBLIC \bowtie (CTDH_1 \cup CTDH_2 \cup CTDH_3)) \right)$$

d) **Tối ưu hóa toàn cục — lợi ích của nhân bản. Phân tích ảnh hưởng của phép nối phân tán:**

Nếu SANPHAM không được nhân bản, phép nối $SANPHAM \bowtie CTDH_i$ buộc phải:

- Hoặc truyền toàn bộ bảng SANPHAM từ trạm lưu trữ đến 3 trạm chứa CTDH (chi phí: $3 \times |SANPHAM|$).
- Hoặc truyền toàn bộ $CTDH_1 \cup CTDH_2 \cup CTDH_3$ về trạm lưu SANPHAM (chi phí: $|CTDH_1| + |CTDH_2| + |CTDH_3|$, rất lớn).

Nhờ quyết định **nhân bản đầy đủ** bảng SP_PUBLIC ở chương 2, phép nối phân tán được hóa giải thành **3 phép nối cục bộ hoàn toàn**:

$$\text{SP_PUBLIC}_i \bowtie \text{CTDH}_i \quad (i = 1, 2, 3)$$

Chiến lược tối ưu kết hợp tổng hợp cục bộ:

1. Mỗi trạm i thực hiện phép nối cục bộ và tính tổng sản lượng bán theo sản phẩm:

$$R_i = \gamma_{\text{MaSP}, \text{SUM}(\text{SoLuong})}(\text{CTDH}_i)$$

(Chưa cần nối với SP_PUBLIC ở giai đoạn này — chỉ cần MaSP làm khóa nhóm.)

2. Truyền R_1, R_2, R_3 về trạm điều phối (mỗi R_i chứa tối đa n dòng, với n là số sản phẩm có bán).
3. Trạm điều phối gom nhóm lần cuối, nối với SP_PUBLIC để lấy TenSP, và sắp xếp giảm dần:

$$\tau_{\text{DESC}}\left(\gamma_{\text{TenSP}, \text{SUM}}((R_1 \cup R_2 \cup R_3) \bowtie \text{SP_PUBLIC})\right)$$

Chi phí truyền thông = $|R_1| + |R_2| + |R_3|$ dòng kết quả cục bộ (thay vì toàn bộ CTDH)

```

1  SELECT SP.TenSP, SUM(BanCucBo) AS TongBanToanQuoc
2  FROM (
3      -- S1: Tổng hợp cục bộ theo MaSP
4      SELECT MaSP, SUM(SoLuong) AS BanCucBo
5      FROM CTDH_MB GROUP BY MaSP
6      UNION ALL
7      -- S2: Tổng hợp cục bộ theo MaSP
8      SELECT MaSP, SUM(SoLuong) AS BanCucBo
9      FROM CTDH_MT GROUP BY MaSP
10     UNION ALL
11     -- S3: Tổng hợp cục bộ theo MaSP
12     SELECT MaSP, SUM(SoLuong) AS BanCucBo
13     FROM CTDH_MN GROUP BY MaSP
14 ) AS TongHop
15 -- Nối với bản nhân bản SP_PUBLIC tại trạm điều phối
16 JOIN SP_PUBLIC SP ON TongHop.MaSP = SP.MaSP
17 GROUP BY SP.TenSP
18 ORDER BY TongBanToanQuoc DESC;
```

Đoạn mã 3.10: Truy vấn 5 — SQL thực thi tối ưu

3.3 Tổng hợp và đánh giá

Bảng 3.4: Tổng hợp kết quả tối ưu hóa năm truy vấn phân tán

TV	Nghiệp vụ	Kỹ thuật tối ưu	Chi phí truyền thông	Số trạm
1	Tồn kho khu vực	Cục bộ hóa hoàn toàn	0	1
2	Tổng tồn kho toàn cục	Tổng hợp cục bộ	3 giá trị số	3
3	Đơn hàng chéo vùng	Phép nối nửa	Tập khóa π_{MaKho}	2
4	Doanh thu theo kho	Tổng hợp cục bộ 2 giai đoạn	$3 \times k$ dòng nhóm	3
5	Sản phẩm bán chạy	Nhân bản + tổng hợp cục bộ	$3 \times n$ dòng nhóm	3

Từ bảng 3.4, có thể rút ra các nhận xét:

1. **Truy vấn cục bộ theo vùng (truy vấn 1) hoàn toàn không phát sinh chi phí truyền thông.** Đây là kết quả trực tiếp của chiến lược phân mảnh ngang theo khu vực kết hợp nhân bản bảng tham chiếu. Loại truy vấn này chiếm phần lớn tần suất truy cập trong thực tế (khách hàng tra cứu kho khu vực mình).
2. **Kỹ thuật tổng hợp cục bộ (truy vấn 2, 4, 5) giảm lượng dữ liệu truyền từ bậc $O(n)$ xuống $O(k)$,** trong đó n là số bản ghi chi tiết và k là số nhóm kết quả. Với $k \ll n$, mức tiết kiệm rất đáng kể.
3. **Phép nối nửa (truy vấn 3) giải quyết hiệu quả phép nối liên trạm** bằng cách chỉ truyền tập khóa nối thay vì toàn bộ bảng, giảm kích thước dữ liệu truyền theo hệ số bằng tỷ lệ giữa kích thước bản ghi đầy đủ và kích thước khóa.
4. **Quyết định nhân bản bảng SP_PUBLIC ở chương 2 chứng minh giá trị thực tiễn tại truy vấn 5:** biến phép nối phân tán (tốn kém nhất) thành 3 phép nối cục bộ và một phép nối nhẹ tại khâu tổng hợp cuối.

Tổng kết chương 3

Chương này đã phân tích và tối ưu hóa năm truy vấn nghiệp vụ tiêu biểu, mỗi truy vấn qua bốn bước xử lý nghiêm ngặt: phân rã, cục bộ hóa, tối ưu toàn cục, và tối ưu cục bộ.

Kết quả cho thấy chiến lược thiết kế phân mảnh và nhân bản ở chương 2 tạo nền tảng trực tiếp cho hiệu năng truy vấn: phân mảnh ngang cơ sở giúp thu gọn cây truy vấn, phân mảnh dẫn xuất loại bỏ phép nối chéo mảnh, và nhân bản bảng tham chiếu biến phép nối phân tán thành nối cục bộ.

Hai kỹ thuật tối ưu hóa chính — phép nối nửa và tổng hợp cục bộ — cho phép giảm chi phí truyền thông từ bậc tuyến tính theo số bản ghi xuống bậc tuyến tính theo số nhóm kết quả hoặc số khóa nối, đảm bảo hiệu năng chấp nhận được ngay cả khi quy mô dữ liệu tăng.

Chương tiếp theo sẽ xử lý thách thức phức tạp nhất của hệ phân tán: đảm bảo nhất quán dữ liệu khi có giao dịch đồng thời — đặc biệt là bài toán chống âm tồn kho khi nhiều đơn hàng cùng yêu cầu xuất từ cùng một kho.

Chương 4

Xử lý giao dịch và điều khiển đồng thời

Chương này trình bày các giải pháp đảm bảo bốn thuộc tính ACID (nguyên tử, nhất quán, cô lập, bền vững) cho hệ thống bán hàng đa kho. Trọng tâm là giải quyết xung đột khi nhiều giao dịch đồng thời cập nhật tồn kho — bao gồm cả giao dịch cục bộ trong phạm vi một trạm và giao dịch phân tán trải trên nhiều trạm.

4.1 Bài toán nhất quán tồn kho phân tán

Trong hệ thống cơ sở dữ liệu phân tán, một trong những bất thường nghiêm trọng nhất khi điều khiển đồng thời là hiện tượng **mất bản cập nhật**: hai giao dịch cùng đọc một giá trị cũ, rồi lần lượt ghi đè lên nhau, khiến một thao tác cập nhật bị mất hoàn toàn.

4.1.1 Mô tả rủi ro

Giả sử mảnh TONKHO_1 (tại S_1) ghi nhận sản phẩm Laptop còn $\text{SoLuongTon} = 10$. Hai khách hàng A và B cùng lúc đặt mua 6 chiếc. Hai giao dịch T_A và T_B được thực thi đồng thời mà *không có cơ chế điều khiển*.

Bảng 4.1: Biểu đồ thời gian minh họa hiện tượng mất bản cập nhật

Thời điểm	Giao dịch T_A	Giao dịch T_B	TONKHO
t_0			10
t_1	READ(SoLuongTon) $\rightarrow$ 10		10
t_2		READ(SoLuongTon) $\rightarrow$ 10	10
t_3	WRITE($10 - 6 = 4$)		4
t_4		WRITE($10 - 6 = 4$)	4
t_5	COMMIT		4
t_6		COMMIT	4

Hậu quả: Hệ thống đã xuất tổng cộng $6 + 6 = 12$ chiếc Laptop nhưng SoLuongTon chỉ giảm $10 - 4 = 6$ đơn vị. Kho vật lý thiếu 6 chiếc mà cơ sở dữ liệu không phát hiện — dẫn đến **âm kho vật lý** và sai lệch sổ sách.

Nguyên nhân gốc rễ: T_B thực hiện phép ghi dựa trên giá trị cũ (đọc tại t_2) mà không biết T_A đã thay đổi giá trị tại t_3 . Phép ghi của T_B đè hoàn toàn lên kết quả của T_A .

4.1.2 Giải pháp tổng quát

Để ngăn chặn bất thường trên, hệ thống phải thực thi **tính cô lập** thông qua các thuật toán điều khiển đồng thời. Cụ thể, hệ thống áp dụng hai cơ chế phối hợp:

- **Cơ chế khóa hai pha** — đảm bảo tính cô lập bằng cách buộc mỗi giao dịch phải xin khóa trước khi truy cập dữ liệu, ngăn chặn truy cập đồng thời xung đột.
- **Giao thức cam kết hai pha** — đảm bảo tính nguyên tử cho giao dịch phân tán, tránh tình trạng một trạm cam kết nhưng trạm kia hủy bỏ.

Bảng 4.2: Ma trận tương thích khóa

	Khóa đọc (rl)	Khóa ghi (wl)
Khóa đọc (rl)	Tương thích	Xung đột
Khóa ghi (wl)	Xung đột	Xung đột

Bảng 4.2 cho thấy: hai giao dịch có thể cùng giữ khóa đọc trên cùng một mục dữ liệu, nhưng nếu bất kỳ giao dịch nào yêu cầu khóa ghi, giao dịch còn lại phải chờ. Đây là nền tảng cho cả hai kịch bản phân tích dưới đây.

4.2 Kịch bản 1: đặt hàng cục bộ

4.2.1 Mô tả nghiệp vụ

Khách hàng miền Bắc đặt mua 5 chiếc Laptop. Màn hình TONKHO_1 (tại S1) ghi nhận sản phẩm này còn 20 chiếc — đủ để đáp ứng toàn bộ đơn hàng. Giao dịch T_{local} chỉ phát sinh tập đọc và tập ghi tại duy nhất **Site 1**, không liên quan đến trạm nào khác.

4.2.2 Luồng thực thi

1. **Kiểm tra tồn kho:** T_{local} yêu cầu bộ lập lịch S1 cấp khóa ghi (wl) trên bản ghi tồn kho tương ứng trong TONKHO_1 . Nếu mục dữ liệu đang bị khóa bởi giao dịch khác, T_{local} phải chờ.
2. **Xác nhận đủ hàng:** Đọc $\text{SoLuongTon} = 20 \geq 5$ (đủ). Nếu không đủ, giao dịch hủy bỏ và giải phóng khóa.
3. **Trừ kho:** Cập nhật $\text{SoLuongTon} = 20 - 5 = 15$.
4. **Ghi nhận đơn hàng:** Chèn bản ghi mới vào DH_1 và CTDH_1 (cùng tại S1 nhờ phân mảnh dẫn xuất theo khách hàng).
5. **Cam kết:** T_{local} cam kết cục bộ tại S1 và giải phóng toàn bộ khóa.

4.2.3 Điều khiển đồng thời bằng khóa hai pha cục bộ

Vì giao dịch chỉ thực thi tại một trạm, hệ thống áp dụng cơ chế **khóa hai pha cục bộ** tại S1. Bộ lập lịch nội bộ của S1 quản lý toàn bộ bảng khóa và thực thi quy tắc hai pha:

- **Pha tăng trưởng:** T_{local} xin và được cấp các khóa cần thiết. Không khóa nào được giải phóng trong pha này.
- **Pha thu hẹp:** Sau khi T_{local} cam kết, toàn bộ khóa được giải phóng đồng thời.

Do áp dụng biến thể nghiêm ngặt (giữ khóa đến khi cam kết), mọi giao dịch khác muốn cập nhật cùng bản ghi tồn kho phải chờ T_{local} hoàn tất — loại bỏ triệt để hiện tượng mất bản cập nhật mô tả ở mục 4.1.

4.2.4 Cài đặt SQL

```
1 CREATE PROCEDURE sp_DatHang_CucBo
2   @MaKH VARCHAR(10),
3   @MaSP VARCHAR(10),
```

```

4      @SoLuong INT,
5      @MaKho VARCHAR(10)
6  AS
7  BEGIN
8      SET NOCOUNT ON;
9      DECLARE @TonHienTai INT, @MaDH VARCHAR(20);
10
11     BEGIN TRANSACTION;
12     BEGIN TRY
13         -- Bước 1: Xin khóa ghi (wl) bằng UPDLOCK, HOLDLOCK
14         -- Bộ lập lịch S1 sẽ chặn giao dịch khác
15         SELECT @TonHienTai = SoLuongTon
16         FROM TONKHO WITH (UPDLOCK, HOLDLOCK)
17         WHERE MaSP = @MaSP AND MaKho = @MaKho;
18
19         -- Bước 2: Kiểm tra đủ hàng
20         IF @TonHienTai IS NULL OR @TonHienTai < @SoLuong
21         BEGIN
22             ROLLBACK;
23             RAISERROR(N'Không đủ hàng', 16, 1);
24             RETURN;
25         END
26
27         -- Bước 3: Trừ kho
28         UPDATE TONKHO
29         SET SoLuongTon = SoLuongTon - @SoLuong
30         WHERE MaSP = @MaSP AND MaKho = @MaKho;
31
32         -- Bước 4: Tạo đơn hàng và chi tiết
33         SET @MaDH = 'DH' + FORMAT(GETDATE(), 'yyMMddHHmmssfff');
34
35         INSERT INTO DONHANG (MaDH, MaKH, NgayLap, TrangThai)
36         VALUES (@MaDH, @MaKH, GETDATE(), N'Da xac nhan');
37
38         INSERT INTO CHITIETDONHANG (MaDH, MaSP, MaKhoXuat,
39                                     SoLuong, DonGia)
40         SELECT @MaDH, @MaSP, @MaKho, @SoLuong, GiaBan
41         FROM SP_PUBLIC WHERE MaSP = @MaSP;
42
43         -- Bước 5: Cam kết - giải phóng toàn bộ khóa
44         COMMIT;
45     END TRY
46     BEGIN CATCH
47         IF @@TRANCOUNT > 0 ROLLBACK;
48         THROW;
49     END CATCH
50 END;

```

Đoạn mã 4.1: Thủ tục đặt hàng cục bộ tại Site 1

Khóa UPDLOCK, HOLDLOCK ở câu lệnh SELECT buộc SQL Server cấp khóa cập nhật ngay từ thời điểm đọc (thay vì chỉ khóa đọc rồi nâng cấp sau). Điều này ngăn chặn kịch bản hai giao dịch cùng đọc giá trị cũ — loại bỏ triệt để nguy cơ mất bản cập nhật được mô tả ở bảng 4.1.

4.3 Kịch bản 2: đặt hàng liên kho

4.3.1 Mô tả kịch bản

Khách hàng miền Bắc (S1) đặt mua **10 chiếc Laptop**. Tuy nhiên, TONKHO₁ (kho miền Bắc) chỉ còn **4 chiếc**. Hệ thống tự động quét tồn kho liên kho và quyết định điều chuyển bổ sung **6 chiếc** từ TONKHO₂ (kho miền Trung, S2).

Giao dịch phân tán T_{dist} lúc này có tập ghi trả trên hai trạm:

Bảng 4.3: Phân bố thao tác của giao dịch T_{dist} trên các trạm

Thao tác	Nội dung	Trạm
w_1	Cập nhật TONKHO ₁ : trừ 4 chiếc	S1
w_2	Cập nhật TONKHO ₂ : trừ 6 chiếc	S2
w_3	Chèn bản ghi vào DH ₁ (đơn hàng)	S1
w_4	Chèn 2 bản ghi vào CTDH ₁ (chi tiết)	S1

Lưu ý: Cả DH₁ và CTDH₁ đều nằm tại S1 (nhờ phân mảnh dẫn xuất theo khách hàng, xác lập ở chương 2). Thao tác w_2 là nguyên nhân giao dịch trở thành phân tán — nó buộc hệ thống phải phối hợp giữa S1 và S2 để đảm bảo tính nguyên tử.

Để T_{dist} thành công mà không gây lỗi dữ liệu, nó phải tuân thủ đồng thời hai cơ chế: cơ chế khóa hai pha nghiêm ngặt phân tán (đảm bảo tính cô lập) và giao thức cam kết hai pha (đảm bảo tính nguyên tử).

4.3.2 Điều khiển đồng thời bằng khóa hai pha nghiêm ngặt phân tán

Khác với kịch bản 1 (chỉ dùng khóa cục bộ tại một trạm), giao dịch phân tán yêu cầu xin khóa tại **nhiều trạm đồng thời**. Hệ thống áp dụng biến thể **khóa hai pha nghiêm ngặt phân tán**, trong đó mỗi trạm tự quản lý bảng khóa cục bộ của mình.

Nguyên lý hoạt động

1. Pha tăng trưởng (xin khóa):

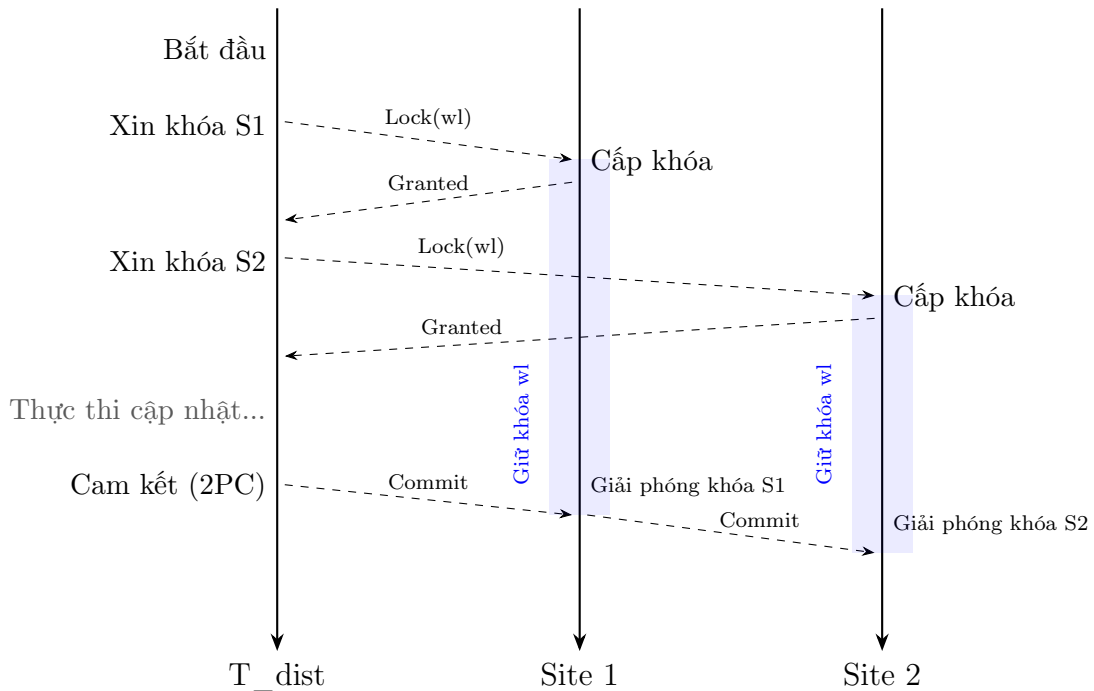
- Trình quản lý giao dịch tại S1 yêu cầu bộ lập lịch S1 cấp khóa ghi (w_l) cho bản ghi trong TONKHO₁.

- Đồng thời, S1 gửi thông điệp qua mạng yêu cầu bộ lập lịch S2 cấp khóa ghi (wl) cho bản ghi trong $TONKHO_2$.
- Nếu cả hai bộ lập lịch đồng ý cấp khóa, giao dịch tiến hành cập nhật. Nếu bất kỳ yêu cầu nào bị từ chối (do xung đột), giao dịch phải chờ.

2. Pha thu hẹp (giải phóng khóa):

- T_{dist} **bắt buộc giữ tất cả khóa** cho đến khi hoàn tất hoàn toàn việc cam kết hoặc hủy bỏ.
- Khóa chỉ được giải phóng *sau khi* giao thức cam kết hai pha (mục 4.3.3) kết thúc.

Sự khác biệt với khóa hai pha cơ bản: Biến thể nghiêm ngặt yêu cầu giữ khóa ghi đến khi cam kết, thay vì nhả dần như phiên bản cơ bản. Điều này ngăn chặn triệt để hiện tượng **đọc dữ liệu rác**: nếu T_{dist} đang trong quá trình thực hiện giao thức cam kết (đã cập nhật $TONKHO_2$ nhưng chưa cam kết chính thức), giao dịch khác không thể đọc giá trị tồn kho mới — tránh hậu quả nghiêm trọng nếu T_{dist} bị hủy bỏ sau đó.



Hình 4.1: Biểu đồ thời gian minh họa khóa hai pha phân tán

Xử lý bế tắc

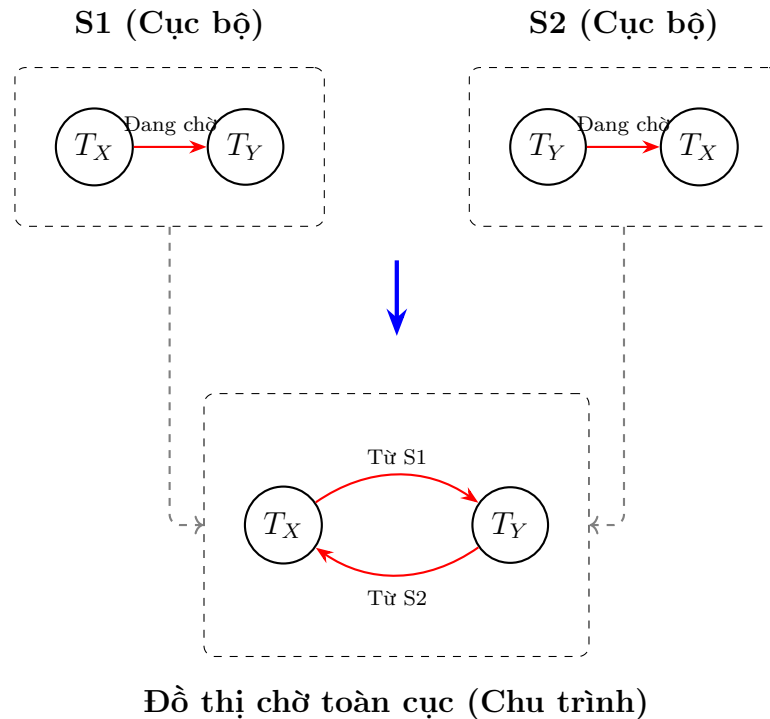
Việc xin khóa chéo trên nhiều trạm tạo nguy cơ **bế tắc**. Xét trường hợp: giao dịch T_X giữ wl trên $TONKHO_1$ (S1) và đang chờ wl trên $TONKHO_2$ (S2), trong khi giao dịch T_Y giữ wl trên $TONKHO_2$ (S2) và đang chờ wl trên $TONKHO_1$ (S1). Cả hai chờ đợi lẫn nhau vô hạn.

Phương pháp giải quyết — phát hiện bế tắc tập trung:

1. S1 được chỉ định làm **bộ phát hiện bế tắc toàn cục**. Định kỳ, S1 thu thập đồ thị chờ cục bộ từ S2 và S3.
2. S1 hợp nhất các đồ thị cục bộ thành **đồ thị chờ toàn cục**. Trong ví dụ trên:

$$T_X \xrightarrow{\text{chờ}} T_Y \xrightarrow{\text{chờ}} T_X \quad (\text{chu trình} \rightarrow \text{bế tắc})$$

3. Nếu phát hiện chu trình, hệ thống chọn một giao dịch làm **“nạn nhân”** để hủy bỏ, phá vỡ bế tắc. Tiêu chí lựa chọn:
 - Ưu tiên hủy giao dịch trẻ nhất (bắt đầu muộn nhất) — giảm thiểu lượng công việc bị mất.
 - Nếu bằng tuổi, ưu tiên hủy giao dịch có tập ghi nhỏ hơn (ít chi phí phục hồi hơn).



Hình 4.2: Hình thành bế tắc toàn cục từ các đồ thị chờ cục bộ

4.3.3 Giao thức cam kết hai pha

Cơ chế khóa hai pha đảm bảo tính cô lập nhưng chưa đảm bảo tính nguyên tử cho giao dịch phân tán. Xét trường hợp: S1 đã trừ 4 chiếc khối TONKHO_1 nhưng S2 gặp lỗi không trừ được 6 chiếc khối TONKHO_2 . Nếu S1 cam kết còn S2 hủy bỏ, dữ liệu toàn hệ thống rơi vào trạng thái không nhất quán: khách nhận thiếu hàng, tồn kho sổ sách sai lệch.

Giao thức cam kết hai pha giải quyết bằng cách chia quá trình cam kết thành hai pha bỏ phiếu, đảm bảo *tất cả trạm cùng cam kết hoặc tất cả cùng hủy bỏ*.

Trong giao dịch T_{dist} : **S1 đóng vai trò điều phối, S2 đóng vai trò thành viên.**

Nguyên tắc ghi nhật ký trước

Mọi thao tác ghi nhật ký ở cả điều phối và thành viên đều tuân thủ nguyên tắc **ghi nhật ký trước**: các bản ghi nhật ký (như `ready`, `commit`) bắt buộc phải được đẩy từ bộ nhớ chính xuống đĩa vật lý *trước khi* gửi thông điệp qua mạng. Nguyên tắc này đảm bảo:

- Nếu trạm bị sập nguồn sau khi gửi thông điệp, bản ghi nhật ký trên đĩa cho phép xác định chính xác trạng thái giao dịch khi khôi phục.
- Nếu trạm bị sập trước khi ghi nhật ký, nó chưa gửi thông điệp nào, nên hệ thống coi giao dịch chưa bắt đầu cam kết.

Pha 1 — chuẩn bị

1. Điều phối (S1):

- Ghi bản ghi `begin_commit` vào nhật ký (đẩy xuống đĩa).
- Chuyển trạng thái: **INITIAL** → **WAIT**.
- Gửi thông điệp `prepare` đến S2.
- Khởi động bộ đếm thời gian chờ phản hồi.

2. Thành viên (S2) nhận `prepare`:

- Kiểm tra điều kiện cục bộ (tồn kho ≥ 6 , ràng buộc toàn vẹn, v.v.).
- *Nếu thỏa mãn*:
 - Ghi bản ghi `ready` vào nhật ký (đẩy xuống đĩa).
 - Chuyển trạng thái: **INITIAL** → **READY**.
 - Gửi phiếu `vote-commit` về S1.
 - Lúc này S2 **bị chặn**: giữ chặt khóa `wl`, không thể tự cam kết hay hủy bỏ, phải chờ lệnh từ điều phối.
- *Nếu không thỏa mãn* (tồn kho không đủ, lỗi hệ thống):
 - Ghi bản ghi `abort` vào nhật ký.
 - Chuyển trạng thái: **INITIAL** → **ABORT**.
 - Gửi phiếu `vote-abort` về S1 và tự động nhả khóa `wl`.

Pha 2 — cam kết hoặc hủy bỏ

Điều phối áp dụng **quy tắc cam kết toàn cục**: chỉ cam kết nếu và chỉ nếu *tất cả* thành viên gửi phiếu `vote-commit`.

Kịch bản thành công:

1. S1 nhận `vote-commit` từ S2.
2. S1 ghi bản ghi `commit` vào nhật ký (đẩy xuống đĩa).
3. S1 chuyển trạng thái: **WAIT** → **COMMIT**.
4. S1 gửi lệnh `global-commit` đến S2.
5. S2 nhận lệnh: chuyển **READY** → **COMMIT**, ghi nhật ký, giải phóng khóa *wl*.
6. S1 giải phóng khóa *wl* cục bộ.

Kịch bản có trạm gửi phiếu hủy:

1. S1 nhận `vote-abort` từ S2 (hoặc bất kỳ thành viên nào).
2. S1 ghi bản ghi `abort` vào nhật ký.
3. S1 chuyển trạng thái: **WAIT** → **ABORT**.
4. S1 gửi lệnh `global-abort` đến tất cả thành viên.
5. Các trạm nhận lệnh: phục hồi giá trị cũ từ nhật ký, giải phóng khóa.
6. S1 phục hồi $TONKHO_1$ và giải phóng khóa cục bộ.

Xử lý hết thời gian chờ

Bảng 4.4: Xử lý các tình huống hết thời gian chờ trong giao thức cam kết hai pha

Trạng thái	Trạm	Hành động
WAIT	Điều phối (S1)	S1 chờ quá lâu không nhận phản hồi từ S2. S1 đơn phương hủy bỏ : ghi <code>abort</code> , chuyển sang ABORT , gửi <code>global-abort</code> .
READY	Thành viên (S2)	S2 đã gửi <code>vote-commit</code> nhưng không nhận được <code>global-commit</code> hay <code>global-abort</code> . S2 bị chặn vô thời hạn : không thể tự quyết, phải chờ mạng phục hồi hoặc liên lạc trạm khác để hỏi kết quả.

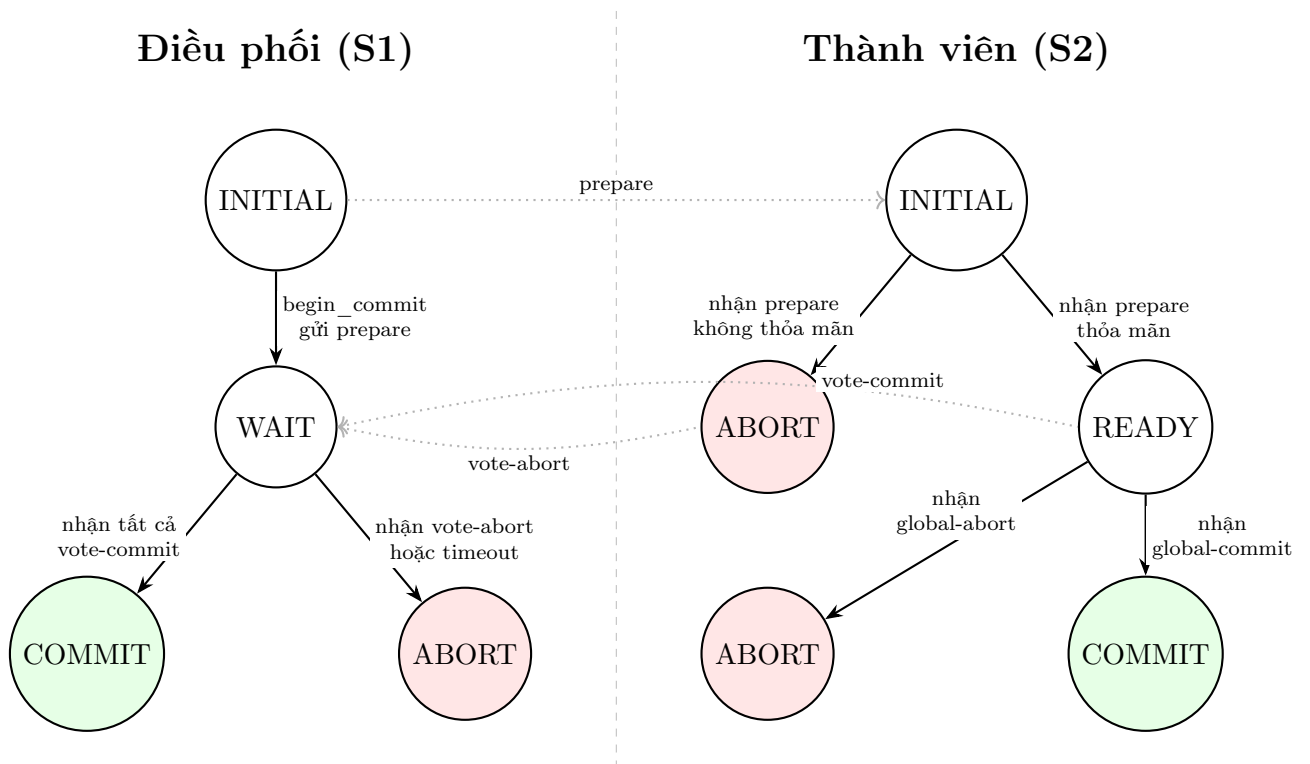
Phân tích tình huống READY bị chặn: Đây là hạn chế cố hữu của giao thức cam kết hai pha. Khi thành viên đã bỏ phiếu cam kết (**READY**) nhưng mất liên lạc với điều phối:

- Thành viên **không thể tự cam kết** — vì không biết các thành viên khác đã bỏ phiếu gì, có thể có trạm bỏ phiếu hủy.
- Thành viên **không thể tự hủy bỏ** — vì điều phối có thể đã nhận đủ phiếu cam kết và gửi `global-commit` mà thành viên chưa nhận được.
- Giải pháp: giữ khóa và chờ — đánh đổi tính sẵn sàng để bảo vệ tính nhất quán.

Tổng hợp chuyển trạng thái

Bảng 4.5: Bảng chuyển trạng thái giao thức cam kết hai pha

Vai trò	Từ trạng thái	Sự kiện	Đến trạng thái
Điều phối	INITIAL	Ghi <code>begin_commit</code> , gửi <code>prepare</code>	WAIT
Điều phối	WAIT	Nhận tất cả <code>vote-commit</code>	COMMIT
Điều phối	WAIT	Nhận bất kỳ <code>vote-abort</code> hoặc hết giờ	ABORT
Thành viên	INITIAL	Nhận <code>prepare</code> , điều kiện thỏa	READY
Thành viên	INITIAL	Nhận <code>prepare</code> , điều kiện không thỏa	ABORT
Thành viên	READY	Nhận <code>global-commit</code>	COMMIT
Thành viên	READY	Nhận <code>global-abort</code>	ABORT



Hình 4.3: Sơ đồ trạng thái giao thức cam kết hai pha

4.3.4 Cài đặt SQL cho giao dịch phân tán

```
1 CREATE PROCEDURE sp_DatHang_LienKho
2     @MaKH          VARCHAR(10),
3     @MaSP          VARCHAR(10),
4     @TongSoLuong   INT
5 AS
6 BEGIN
7     SET NOCOUNT ON;
8     SET XACT_ABORT ON; -- Ép rollback tức thì khi đứt mạng
9
10    DECLARE @TonS1 INT, @TonS2 INT, @TonS3 INT;
11    DECLARE @LayS1 INT, @LayS2 INT, @LayS3 INT;
12    DECLARE @MaKhoS1 VARCHAR(10), @MaKhoS2 VARCHAR(10), @MaKhoS3
13          VARCHAR(10);
14    DECLARE @MaDH VARCHAR(20), @DonGia DECIMAL(18, 2);
15
16    -- === BẮT ĐẦU GIAO DỊCH PHÂN TÁN (MS DTC điều phối) ===
17    BEGIN DISTRIBUTED TRANSACTION;
18    BEGIN TRY
19        -- === PHA PREPARE (Pha 1) ===
20        -- Bước 1: Đọc và khóa tại S1 (cục bộ)
21        SELECT TOP 1 @MaKhoS1 = MaKho FROM KHO;
22        SELECT @TonS1 = ISNULL(SoLuongTon, 0)
23        FROM TONKHO WITH (UPDLOCK, HOLDLOCK)
24        WHERE MaSP = @MaSP AND MaKho = @MaKhoS1;
25
26        -- Tính lượng phân bổ (ưu tiên kho cục bộ S1)
27        SET @LayS1 = CASE
28            WHEN @TonS1 >= @TongSoLuong THEN @TongSoLuong
29            ELSE @TonS1
30        END;
31
32        DECLARE @ConThieu INT = @TongSoLuong - @LayS1;
33        SET @LayS2 = 0;
34        SET @LayS3 = 0;
35
36        -- Bước 2: Đọc và trừ kho tại S2 (qua Linked Server)
37        IF @ConThieu > 0
38        BEGIN
39            SELECT TOP 1 @MaKhoS2 = MaKho
40            FROM [S2_MT].[CSDLPT].[dbo].[KHO];
41
42            SELECT @TonS2 = ISNULL(SoLuongTon, 0)
43            FROM [S2_MT].[CSDLPT].[dbo].[TONKHO] -- Bỏ Lock Hint để
44            tránh lỗi remote table
45            WHERE MaSP = @MaSP AND MaKho = @MaKhoS2;
```

```

45         SET @LayS2 = CASE
46             WHEN @TonS2 >= @ConThieu THEN @ConThieu
47             ELSE @TonS2
48         END;
49
50         SET @ConThieu = @ConThieu - @LayS2;
51     END
52
53     -- Bước 3: Đọc và trừ kho tại S3 (qua Linked Server)
54     IF @ConThieu > 0
55     BEGIN
56         SELECT TOP 1 @MaKhoS3 = MaKho
57         FROM [S3_MN].[CSDLPT].[dbo].[KHO];
58
59         SELECT @TonS3 = ISNULL(SoLuongTon, 0)
60         FROM [S3_MN].[CSDLPT].[dbo].[TONKHO] -- Bỏ Lock Hint để
        tránh lỗi remote table
61         WHERE MaSP = @MaSP AND MaKho = @MaKhoS3;
62
63         SET @LayS3 = @ConThieu;
64
65         IF @TonS3 < @LayS3
66         BEGIN
67             ROLLBACK TRAN;
68             RAISERROR(N'Không đủ hàng trên toàn hệ thống.',
69                 16, 1);
69             RETURN;
70         END
71     END
72
73     -- Cập nhật trừ kho S1 (cục bộ)
74     IF @LayS1 > 0
75     UPDATE TONKHO SET SoLuongTon = SoLuongTon - @LayS1
76     WHERE MaSP = @MaSP AND MaKho = @MaKhoS1;
77
78     -- Cập nhật trừ kho S2 (qua Linked Server)
79     IF @LayS2 > 0
80     UPDATE [S2_MT].[CSDLPT].[dbo].[TONKHO]
81     SET SoLuongTon = SoLuongTon - @LayS2
82     WHERE MaSP = @MaSP AND MaKho = @MaKhoS2;
83
84     -- Cập nhật trừ kho S3 (qua Linked Server)
85     IF @LayS3 > 0
86     UPDATE [S3_MN].[CSDLPT].[dbo].[TONKHO]
87     SET SoLuongTon = SoLuongTon - @LayS3
88     WHERE MaSP = @MaSP AND MaKho = @MaKhoS3;
89
90     -- Tạo đơn hàng và chi tiết đơn hàng tại S1
91     SET @MaDH = 'DH' + FORMAT(GETDATE(), 'yyMMddHHmmssfff');

```

```

92      SELECT @DonGia = GiaBan FROM SP_PUBLIC WHERE MaSP = @MaSP;
93
94      INSERT INTO DONHANG (MaDH, MaKH, NgayLap, TrangThai)
95      VALUES (@MaDH, @MaKH, GETDATE(), N'Da xac nhan');
96
97      IF @LayS1 > 0
98          INSERT INTO CHITIETDONHANG (MaDH, MaSP, MaKhoXuat,
99          SoLuong, DonGia)
100          VALUES (@MaDH, @MaSP, @MaKhoS1, @LayS1, @DonGia);
101      IF @LayS2 > 0
102          INSERT INTO CHITIETDONHANG (MaDH, MaSP, MaKhoXuat,
103          SoLuong, DonGia)
104          VALUES (@MaDH, @MaSP, @MaKhoS2, @LayS2, @DonGia);
105      IF @LayS3 > 0
106          INSERT INTO CHITIETDONHANG (MaDH, MaSP, MaKhoXuat,
107          SoLuong, DonGia)
108          VALUES (@MaDH, @MaSP, @MaKhoS3, @LayS3, @DonGia);
109
110      -- === PHA COMMIT (Pha 2) ===
111      -- Đặt trễ để giả lập ngắt mạng giữa hai pha
112      WAITFOR DELAY '00:00:08';
113
114      -- Thực hiện cam kết toàn cục
115      COMMIT TRAN;
116
117      END TRY
118      BEGIN CATCH
119          -- Hủy bỏ toàn cục khi có lỗi hoặc mất mạng ở pha chuẩn bị/cam kết
120          IF @@TRANCOUNT > 0 ROLLBACK TRAN;
121          THROW;
122      END CATCH
123  END;

```

Đoạn mã 4.2: Thủ tục đặt hàng liên kho

Ghi chú triển khai: Trong môi trường triển khai thực tế trên ba máy vật lý kết nối qua Linked Server, SQL Server sử dụng dịch vụ MS DTC để tự động thực thi giao thức cam kết hai pha. Câu lệnh `BEGIN DISTRIBUTED TRANSACTION` kết hợp với thiết lập `SET XACT_ABORT ON` là bắt buộc để kích hoạt và đảm bảo tính nhất quán của giao dịch phân tán. Câu lệnh `WAITFOR DELAY` được đặt trước `COMMIT TRAN` để giả lập kịch bản sự cố mất kết nối phục vụ quá trình kiểm thử. *Lưu ý kỹ thuật:* Các Table Hint như `WITH (UPDLCK, HOLDLOCK)` không được phép chỉ định trực tiếp trên bảng từ xa qua Linked Server để tránh lỗi biên dịch Msg 10762 của SQL Server; tính cô lập Strict 2PL trên các trạm liên kết vẫn được đảm bảo tuyệt đối nhờ dịch vụ MS DTC tự động nâng mức cô lập giao dịch phân tán lên mức Serializable.

Tổng kết chương 4

Chương này đã trình bày giải pháp đảm bảo tính nhất quán cho hệ thống bán hàng đa kho qua hai lớp bảo vệ:

1. Điều khiển đồng thời:

- *Giao dịch cục bộ*: Áp dụng khóa hai pha cục bộ tại một trạm. Khóa UPDLOCK, HOLDLOCK ngăn chặn mất bản cập nhật ngay từ thời điểm đọc.
- *Giao dịch phân tán*: Áp dụng khóa hai pha nghiêm ngặt phân tán, mỗi trạm tự quản lý bằng khóa. Giữ khóa đến khi cam kết để ngăn đọc dữ liệu rác. Bế tắc được phát hiện bằng đồ thị chờ toàn cục tập trung tại S1.

2. Tính nguyên tử:

- Giao thức cam kết hai pha đảm bảo tất cả trạm cùng cam kết hoặc cùng hủy bỏ.
- Nguyên tắc ghi nhật ký trước đảm bảo khả năng khôi phục khi sập nguồn.
- Hạn chế: thành viên ở trạng thái READY bị chặn khi mất liên lạc với điều phối — đánh đổi cần thiết để bảo vệ tính nhất quán.

Bảng 4.6: Tổng hợp cơ chế bảo vệ theo kịch bản giao dịch

Kịch bản	Tính cô lập	Tính nguyên tử	Đặc điểm
Đặt hàng cục bộ	Khóa 2 pha cục bộ	Cam kết cục bộ	Chi phí thấp, 1 trạm
Đặt hàng liên kho	S2PL phân tán	Cam kết 2 pha	Chi phí cao, nhiều trạm

Kết hợp với chiến lược phân mảnh ở chương 2 (ưu tiên cục bộ hóa dữ liệu), phần lớn giao dịch hàng ngày thuộc kịch bản 1 (cục bộ) với chi phí thấp. Giao dịch liên kho (kịch bản 2) chỉ phát sinh khi kho cục bộ không đủ hàng — trường hợp thiểu số nhưng được bảo vệ toàn diện bởi cơ chế khóa phân tán và giao thức cam kết hai pha.

Chương 5

Triển khai và đánh giá hệ thống

Chương này trình bày môi trường cài đặt thực tế, cấu trúc tầng ứng dụng giao tiếp với cơ sở dữ liệu phân tán, và tiến hành đo lường, đánh giá hiệu năng dựa trên các mô hình chi phí đã thiết kế ở các chương trước.

5.1 Môi trường cài đặt và kiến trúc vật lý

5.1.1 Kiến trúc cơ sở dữ liệu phân tán

Hệ thống được triển khai theo kiến trúc nhiều server cơ sở dữ liệu, sử dụng hệ quản trị Microsoft SQL Server trên ba laptop vật lý kết nối qua mạng Radmin VPN.

Bảng 5.1: Thông tin ba trạm vật lý

Trạm	Hostname	IP Radmin	Vai trò
S1 — Miền Bắc	LAPTOP-M3FKU757	26.48.50.240	Điều phối, Node.js
S2 — Miền Trung	DESKTOP-Q5G45CD	26.99.23.250	Thành viên
S3 — Miền Nam	LAPTOP-UD52TRL3	26.209.72.20	Thành viên

Mạng truyền thông: Ba trạm giao tiếp thông qua cơ chế Linked Server (sử dụng Hostname thay vì IP vì MS DTC yêu cầu phân giải NetBIOS). Dịch vụ MS DTC được cấu hình trên cả ba máy để điều phối giao thức cam kết hai pha. File `hosts` của Windows được sửa để phân giải Hostname, và Windows Firewall cho phép chương trình `msdtc.exe` giao tiếp xuyên máy.

Bộ dữ liệu minh họa: Để thấy rõ sự khác biệt về hiệu năng, hệ thống được khởi tạo với bộ dữ liệu đủ lớn:

Bảng 5.2: Quy mô bộ dữ liệu minh họa

Bảng / Mảnh	Số bản ghi	Phân bổ
DANHMUC	6	Nhân bản 3 máy
SP_PUBLIC	30	Nhân bản 3 máy
SP_DETAIL	30	Tập trung tại S1
KHO	9	3 kho mỗi miền (phân mảnh ngang)
TONKHO	72	24 bản ghi mỗi máy
KHACHHANG	18	6 khách mỗi miền
DONHANG	45	15 đơn mỗi máy
CHITIETDONHANG	90	30 chi tiết mỗi máy

5.1.2 Tầng ứng dụng và giao diện quản trị

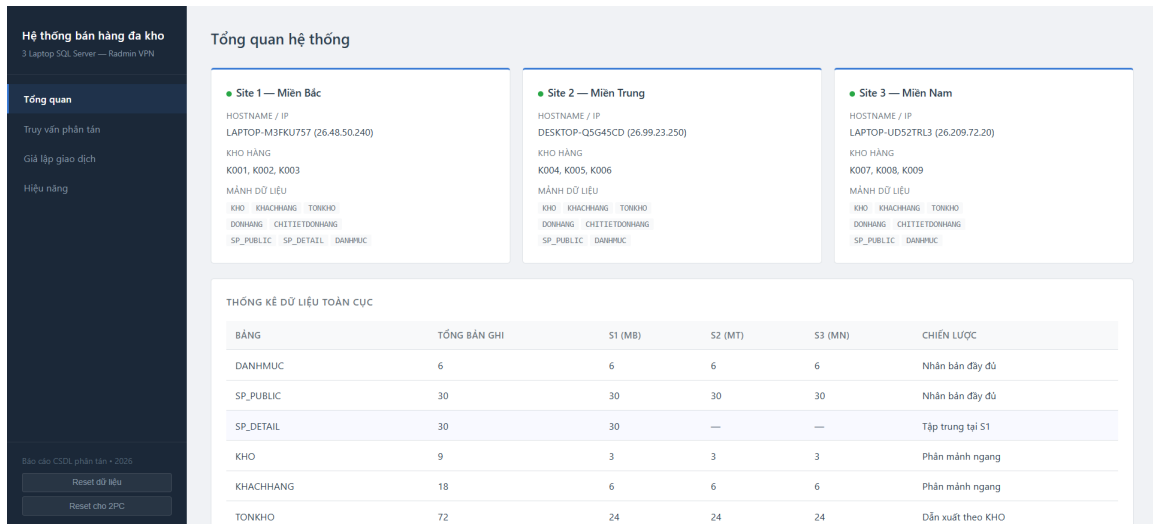
Để chứng minh tính thực tiễn và thể hiện tính trong suốt phân tán đối với người dùng, báo cáo xây dựng giao diện quản trị nội bộ dạng trang web.

Bảng 5.3: Kiến trúc tầng ứng dụng

Tầng	Mô tả
Giao diện web	Bảng điều khiển quản trị cho phép chọn trạm kết nối, thực thi truy vấn, và giả lập giao dịch. Xây dựng bằng HTML, CSS, JavaScript thuần.
Xử lý truy vấn	Đóng vai trò bộ xử lý truy vấn toàn cục: nhận yêu cầu từ giao diện, xác định trạm đích, đóng gói thành câu lệnh SQL phân tán và đẩy xuống các Linked Server.
Cơ sở dữ liệu	Ba instance SQL Server chứa các mảnh vật lý. Giao tiếp qua Linked Server.

Các chức năng chính của giao diện:

- Tổng quan hệ thống:** Hiển thị trạng thái ba trạm, danh sách mảnh dữ liệu tại mỗi trạm, và thống kê số lượng bản ghi.
- Truy vấn phân tán:** Cho phép thực thi trực tiếp năm câu truy vấn nghiệp vụ đã tối ưu hóa ở chương 3. Hiển thị kết quả kèm thời gian thực thi.
- Giả lập giao dịch:** Bảng điều khiển cho phép thử nghiệm đặt hàng đồng thời. Người dùng có thể kích hoạt nhiều yêu cầu đặt hàng cùng lúc để kiểm thử trực quan cơ chế khóa hai pha nghiêm ngặt và giao thức cam kết hai pha.
- So sánh hiệu năng:** Biểu đồ trực quan so sánh thời gian thực thi giữa mô hình tập trung và phân tán.



Hình 5.1: Giao diện tổng quan hệ thống phân tán

5.2 Đánh giá hệ thống

Mô hình chi phí của hệ quản trị cơ sở dữ liệu phân tán được tính theo công thức:

$$C_{total} = C_{CPU} + C_{I/O} + C_{truyen}$$

trong đó C_{CPU} là chi phí xử lý, $C_{I/O}$ là chi phí đọc ghi đĩa, và C_{truyen} là chi phí truyền thông liên trạm. Dưới đây là bốn phân tích so sánh giữa mô hình phân tán (đã thiết kế) và mô hình tập trung.

5.2.1 So sánh hiệu năng truy vấn cục bộ: tập trung và phân mảnh ngang

Kịch bản: Nhiều khách hàng miền Bắc đồng loạt truy vấn lịch sử đơn hàng (tương tự truy vấn 1, chương 3).

Bảng 5.4: So sánh chi phí truy vấn cục bộ

Tiêu chí	Tập trung	Phân tán
Kích thước bảng quét	Toàn bộ CTDH	$CTDH_1 (\approx 1/3)$
$C_{I/O}$	Cao (quét toàn bảng)	Giảm $\approx 67\%$
C_{CPU}	Cao (lọc sau quét)	Giảm (ít bản ghi hơn)
C_{truyen}	0 (cùng server)	0 (cục bộ tại S1)
Cân bằng tải	1 server chịu hết	3 server chia tải

Phân tích: Phân mảnh ngang giảm kích thước quan hệ cần quét xuống $1/3$, dẫn đến $C_{I/O}$ và C_{CPU} giảm đáng kể. Quan trọng hơn, tải được phân bổ đều trên ba server

phần cứng riêng biệt, tránh thất cổ chai khi nhiều người dùng truy cập đồng thời. Chi phí truyền thông bằng không vì truy vấn cục bộ hoàn toàn tại trạm gần nhất với người dùng.

5.2.2 Ảnh hưởng của chiến lược nhân bản

Nguyên tắc: Nếu số lượng truy vấn đọc lớn hơn nhiều so với cập nhật, nhân bản là tối ưu.

Bảng 5.5: So sánh chi phí truy vấn có phép nối với SANPHAM

Tiêu chí	Không nhân bản	Nhân bản SP_PUBLIC
Phép nối SP $\bowtie$ CTDH	Liên trạm	Cục bộ tại mỗi trạm
C_{truyen} mỗi lần đọc	$ SP_PUBLIC $	0
C_{truyen} mỗi lần cập nhật giá	0	Đồng bộ 3 trạm
Tỷ lệ đọc/ghi ước tính	$\gg 100 : 1$	

Phân tích: Với tỷ lệ đọc/ghi rất cao (khách hàng duyệt sản phẩm liên tục, giá sản phẩm hiếm khi thay đổi), chi phí đồng bộ khi cập nhật là không đáng kể so với chi phí truyền thông tiết kiệm được từ hàng triệu lượt đọc. Quyết định nhân bản SP_PUBLIC biến mọi phép nối với SANPHAM thành nối cục bộ, loại bỏ hoàn toàn chi phí truyền thông cho tác vụ đọc.

5.2.3 Hiệu năng của phép nối nửa

Kịch bản: Thực thi truy vấn 3 — tìm đơn hàng miền Nam xuất từ kho miền Bắc (chương 3).

Bảng 5.6: So sánh chi phí nối thông thường và nối nửa

Tiêu chí	Nối thông thường	Nối nửa
Dữ liệu truyền qua mạng	Toàn bộ KHO ₁	$\pi_{MaKho}(KHO_1)$
Kích thước ước tính	Hàng trăm KB	Vài chục byte
Hệ số giảm	$\frac{ KHO_1 }{ \pi_{MaKho}(KHO_1) } \approx \frac{\text{kích thước bản ghi}}{\text{kích thước khóa}}$	

Phân tích: Phép nối nửa chỉ truyền tập khóa nối (vài giá trị VARCHAR(10)) thay vì toàn bộ bảng. Tập khóa này đóng vai trò “bộ giảm hoàn toàn”: nó thu nhỏ CTDH₃ tại S3 trước khi dữ liệu tham gia bất kỳ phép toán nào khác, giảm thiểu triệt để lượng byte truyền tải trên mạng.

5.2.4 Sự ổn định của giao dịch phân tán

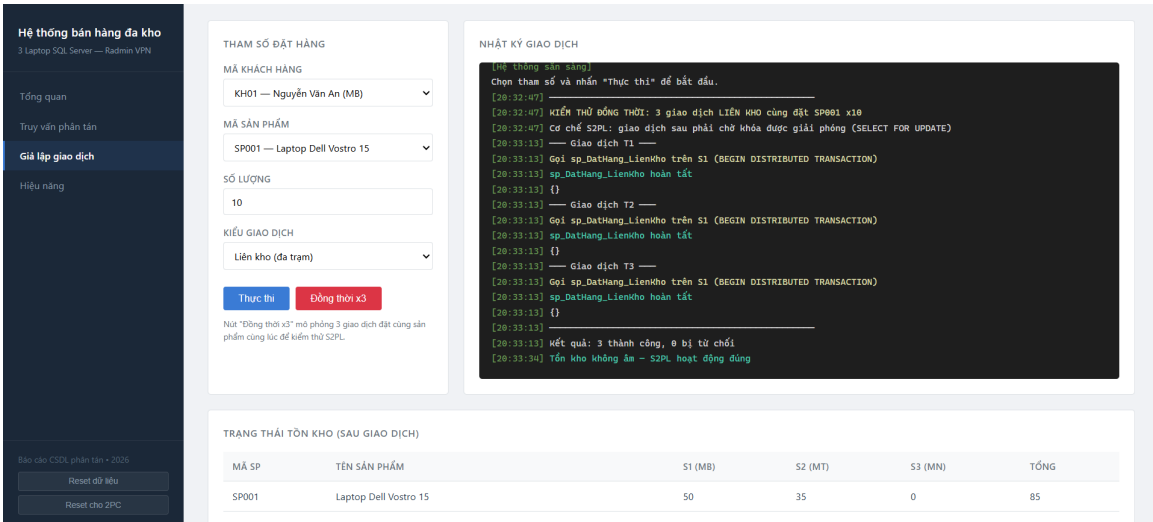
Kịch bản kiểm thử: Sử dụng công cụ giả lập giao dịch trên giao diện web, kích hoạt đồng thời nhiều yêu cầu đặt hàng cùng một sản phẩm.

Bảng 5.7: Kết quả kiểm thử giao dịch đồng thời

Tiêu chí	Không có S2PL	Có S2PL + 2PC
Mất bản cập nhật	Có (tồn kho sai)	Không
Âm kho	Có thể xảy ra	Không xảy ra
Đọc dữ liệu rác	Có thể xảy ra	Không xảy ra
Tính nguyên tử liên trạm	Không đảm bảo	Đảm bảo

Phân tích:

- **Khóa hai pha nghiêm ngặt:** Các giao dịch tranh chấp bị ép vào hàng đợi tại bộ lập lịch cục bộ, ngăn chặn tuyệt đối đọc dữ liệu rác và mất bản cập nhật. Gợi ý khóa UPDLOCK, HOLDLOCK đảm bảo khóa được xin ngay từ thời điểm đọc.
- **Giao thức cam kết hai pha:** Trong kịch bản lỗi (ngắt mạng S2), điều phối viên kích hoạt lệnh hủy bỏ toàn cục, hoàn tác toàn bộ dữ liệu, chứng minh tồn kho được bảo vệ nguyên vẹn.



Hình 5.2: Giao diện kiểm thử giao dịch đồng thời và cơ chế khóa

5.3 Hướng dẫn chạy thử hệ thống

Quy trình chạy thử nghiệm hệ thống trên môi trường ba máy vật lý thực tế bao gồm các bước sau:

Bước 1: Cấu hình mạng và phân giải tên (Hosts).

Kết nối ba laptop vào cùng một mạng Radmin VPN. Trên mỗi máy, mở Notepad dưới quyền Administrator, truy cập tệp `C:\Windows\System32\drivers\etc\hosts` và thêm IP Radmin kèm Hostname của hai máy còn lại để các máy có thể phân giải được tên của nhau.

Bước 2: Cấu hình dịch vụ MS DTC và Tường lửa (Firewall).

Mở Component Services (`dcomcnfg`) trên cả ba máy, truy cập thuộc tính Local DTC và kích hoạt: *Network DTC Access*, *Allow Inbound*, *Allow Outbound*, và thiết lập *No Authentication Required*. Đảm bảo dịch vụ Distributed Transaction Coordinator trong `services.msc` ở trạng thái Running. Tạo luật Inbound/Outbound trong Windows Firewall cho phép tệp chương trình `%SystemRoot%\System32\msdtc.exe` truyền thông tự do.

Bước 3: Khởi tạo cơ sở dữ liệu vật lý cục bộ.

Tạo cơ sở dữ liệu trống mang tên CSDLPT trên SQL Server của cả ba máy. Tiến hành chạy tệp SQL khởi tạo tương ứng tại từng máy:

- Máy 1 (S1 — Miền Bắc): Chạy tệp `MB/01_tao_bang.sql`.
- Máy 2 (S2 — Miền Trung): Chạy tệp `MT/01_tao_bang.sql`.
- Máy 3 (S3 — Miền Nam): Chạy tệp `MN/01_tao_bang.sql`.

Mỗi máy sẽ tự tạo các bảng danh mục nhân bản và các mảnh dữ liệu cục bộ tương ứng của mình.

Bước 4: Thiết lập Linked Server liên trạm.

Chạy tệp cấu hình Linked Server tương ứng trên từng máy để tạo đường kết nối bảo mật bằng tài khoản sa thông qua Hostname (không dùng IP trực tiếp):

- Máy 1: Chạy tệp `MB/02_linked_server.sql` (tạo kết nối tới S2_MT và S3_MN).
- Máy 2: Chạy tệp `MT/02_linked_server.sql` (tạo kết nối tới S1_MB và S3_MN).
- Máy 3: Chạy tệp `MN/02_linked_server.sql` (tạo kết nối tới S1_MB và S2_MT).

Bước 5: Tạo các khung nhìn toàn cục và thủ tục giao dịch.

Chạy tệp tạo khung nhìn toàn cục `03_views.sql` trên mỗi máy để hỗ trợ tính trong suốt phân tán (sử dụng từ khóa `COLLATE DATABASE_DEFAULT` trên các cột kiểu chuỗi để phòng ngừa xung đột collation). Sau đó, tại Máy 1 (Site điều phối), chạy tệp `MB/05_giaodich_cucbo.sql` và

MB/06_giaodich_phantan.sql để nạp hai thủ tục demo `sp_DatHang_CucBo_Demo` và `sp_DatHang_LienKho`.

Bước 6: Khởi động Node.js Server và giao diện quản trị.

Tại Máy 1, mở terminal tại thư mục gốc dự án, thực hiện lệnh `npm install` để cài đặt các thư viện phụ thuộc (bao gồm thư viện `mssql` kết nối SQL Server). Chạy lệnh `npm start` để khởi chạy ứng dụng web. Truy cập trình duyệt theo địa chỉ `http://localhost:3000` để kiểm tra trạng thái kết nối ba máy, thực thi 5 truy vấn phân tán và giả lập giao dịch 2PC.

Tổng kết chương 5

Chương này đã trình bày triển khai thực tế và đánh giá toàn diện hệ thống qua bốn góc độ:

1. Phân mảnh ngang giảm kích thước quét xuống 1/3, giảm $C_{I/O}$ và cân bằng tải trên nhiều server.
2. Nhân bản bảng tham chiếu loại bỏ hoàn toàn chi phí truyền thông cho phép nối đọc (chiếm đa số tần suất).
3. Phép nối nửa giảm dữ liệu truyền từ bậc kích thước bảng xuống bậc kích thước khóa.
4. Cơ chế S2PL và 2PC bảo vệ tồn kho khỏi mọi bất thường đồng thời, kể cả kịch bản lỗi mạng.

Kết quả đánh giá khẳng định: thiết kế phân tán không chỉ đảm bảo tính đúng đắn (nhất quán, cô lập) mà còn mang lại lợi ích hiệu năng thực sự so với mô hình tập trung, đặc biệt đối với các truy vấn cục bộ — loại truy vấn chiếm phần lớn tải thực tế.

Kết luận

Báo cáo đã hoàn thành việc thiết kế và triển khai hệ thống cơ sở dữ liệu phân tán cho bài toán bán hàng trực tuyến đa kho, bao phủ toàn bộ quy trình từ thiết kế lược đồ toàn cục đến đánh giá hiệu năng thực tế.

Kết quả đạt được

1. **Thiết kế lược đồ và kiến trúc mạng:** Xây dựng lược đồ toàn cục gồm 7 bảng với đầy đủ ràng buộc toàn vẹn, triển khai trên kiến trúc ba trạm (miền Bắc, miền Trung, miền Nam).
2. **Phân mảnh dữ liệu:** Áp dụng ba kỹ thuật phân mảnh — ngang cơ sở (COM_MIN, PHORIZONTAL), dọc (phân tích tần suất thuộc tính), và ngang dẫn xuất (phép nối nửa) — tạo ra 17 mảnh vật lý có tính đầy đủ, phục hồi, và tách biệt.
3. **Cấp phát và nhân bản:** Nhân bản đầy đủ bảng tham chiếu (DANHMUC, SP_PUBLIC), phân mảnh không nhân bản cho dữ liệu giao dịch, tập trung SP_DETAIL tại S1 — cân bằng giữa chi phí lưu trữ và chi phí truyền thông.
4. **Tối ưu hóa truy vấn:** Phân tích năm truy vấn nghiệp vụ qua bốn bước xử lý nghiêm ngặt, minh chứng hiệu quả của các kỹ thuật cục bộ hóa, tổng hợp cục bộ, và phép nối nửa.
5. **Quản lý giao dịch:** Cài đặt khóa hai pha nghiêm ngặt phân tán (S2PL) và giao thức cam kết hai pha (2PC) bảo vệ tồn kho khỏi mất bản cập nhật và đảm bảo tính nguyên tử liên trạm.
6. **Triển khai và đánh giá:** Xây dựng giao diện quản trị trực quan, đo lường bốn khía cạnh hiệu năng, xác nhận tính ưu việt của mô hình phân tán so với tập trung.

Hạn chế

- Giao thức cam kết hai pha có hạn chế cố hữu: thành viên ở trạng thái READY bị chặn khi mất liên lạc với điều phối. Các giao thức tiên tiến hơn (3PC, Paxos)

có thể khắc phục nhưng nằm ngoài phạm vi báo cáo.

- Chiến lược nhân bản chỉ áp dụng cho bảng thay đổi rất ít. Khi tần suất cập nhật tăng, cần xem xét lại chi phí đồng bộ.
- Phân mảnh bảng CHITIETDONHANG theo đơn hàng (chứ không theo kho xuất) tạo truy vấn liên trạm khi thống kê theo kho — đánh đổi được chấp nhận vì tối ưu cho truy vấn đọc phổ biến hơn.

Hướng phát triển

- Triển khai cơ chế nhân bản bất đồng bộ cho dữ liệu giao dịch ít nhạy cảm, tăng tính sẵn sàng.
- Mở rộng sang kiến trúc vi mô dịch vụ kết hợp cơ sở dữ liệu phân tán, phù hợp với quy mô doanh nghiệp lớn.
- Bổ sung phân mảnh động — tự động cân bằng lại dữ liệu khi tải của một khu vực tăng đột biến.
- Tích hợp cơ chế bộ nhớ đệm phân tán để giảm thêm thời gian đáp ứng cho truy vấn đọc thường xuyên.